

Deep Residual Learning for Image Recognition

Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun
Microsoft Research
{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

Abstract

Deeper neural networks are more difficult to train. We present a residual learning framework to ease the training of networks that are substantially deeper than those used previously. We explicitly reformulate the layers as learning residual functions with reference to the layer inputs, instead of learning unreferenced functions. We provide comprehensive empirical evidence showing that these residual networks are easier to optimize, and can gain accuracy from considerably increased depth. On the ImageNet dataset we evaluate residual nets with a depth of up to 152 layers—8× deeper than VGG nets [41] but still having lower complexity. An ensemble of these residual nets achieves 3.57% error on the ImageNet test set. This result won the 1st place on the ILSVRC 2015 classification task. We also present analysis on CIFAR-10 with 100 and 1000 layers.

The depth of representations is of central importance for many visual recognition tasks. Solely due to our extremely deep representations, we obtain a 28% relative improvement on the COCO object detection dataset. Deep residual nets are foundations of our submissions to ILSVRC & COCO 2015 competitions¹, where we also won the 1st places on the tasks of ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation.

1. Introduction

Deep convolutional neural networks [22, 21] have led to a series of breakthroughs for image classification [21, 50, 40]. Deep networks naturally integrate low/mid/high-level features [50] and classifiers in an end-to-end multi-layer fashion, and the “levels” of features can be enriched by the number of stacked layers (depth). Recent evidence [41, 44] reveals that network depth is of crucial importance, and the leading results [41, 44, 13, 16] on the challenging ImageNet dataset [36] all exploit “very deep” [41] models, with a depth of sixteen [41] to thirty [16]. Many other non-trivial visual recognition tasks [8, 12, 7, 32, 27] have also

¹<http://image-net.org/challenges/LSVRC/2015/> and <http://mscoco.org/dataset/#detections-challenge2015>.

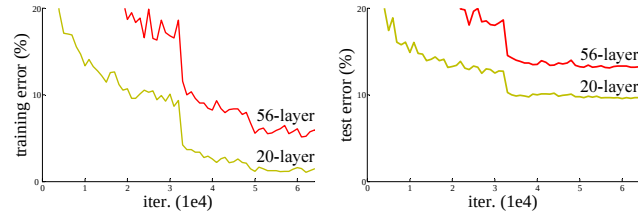


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

greatly benefited from very deep models.

Driven by the significance of depth, a question arises: *Is learning better networks as easy as stacking more layers?* An obstacle to answering this question was the notorious problem of vanishing/exploding gradients [1, 9], which hamper convergence from the beginning. This problem, however, has been largely addressed by normalized initialization [23, 9, 37, 13] and intermediate normalization layers [16], which enable networks with tens of layers to start converging for stochastic gradient descent (SGD) with back-propagation [22].

When deeper networks are able to start converging, a degradation problem has been exposed: with the network depth increasing, accuracy gets saturated (which might be unsurprising) and then degrades rapidly. Unexpectedly, such degradation is *not caused by overfitting*, and adding more layers to a suitably deep model leads to *higher training error*, as reported in [11, 42] and thoroughly verified by our experiments. Fig. 1 shows a typical example.

The degradation (of training accuracy) indicates that not all systems are similarly easy to optimize. Let us consider a shallower architecture and its deeper counterpart that adds more layers onto it. There exists a solution *by construction* to the deeper model: the added layers are *identity* mapping, and the other layers are copied from the learned shallower model. The existence of this constructed solution indicates that a deeper model should produce no higher training error than its shallower counterpart. But experiments show that our current solvers on hand are unable to find solutions that

图像识别深度残差学习

何恺明、张翔宇、任少卿、孙健、微软研究院{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

摘要

层次更深的神经网络往往更难以训练。为此我们提出了一种残差学习框架，用以简化远比以往更深层级的神经网络的训练过程。该框架将各层明确定义为以层输入为参考的待学习残差函数，而非无参考依据的函数。大量实验证据表明，这类残差网络不仅更易于优化，还能通过大幅增加层次来提升分类精度。在ImageNet数据集上，我们对深度高达152层的残差网络进行了测试——8×其深度超过了VGG网络[41]，但复杂度却更低。由这些残差网络构成的集成模型在ImageNet测试集上的误差率为3.57%，这一成绩为2015年ILSVRC竞赛的分类任务赢得了第一名。此外，我们还针对包含100层和1000层的CIFAR-10数据集进行了相关分析。

对于许多视觉识别任务而言，表示层的深度具有至关重要的意义。正是得益于我们极其深厚的表示层，我们在COCO物体检测数据集上的性能提升了28%。深度残差神经网络是我们提交给ILSVRC和COCO 2015竞赛作品的基础¹，在这些竞赛中，我们还在ImageNet检测、ImageNet定位、COCO检测以及COCO分割任务上获得了第一名。

1. 引言

先进的卷积神经网络[22, 21]为图像分类[21, 50, 40]带来了一系列重大突破。这类深度神经网络能够以端到端的多层次方式自然整合低层级、中层级和高层级的特征[50]以及分类器，而特征的“层级”数量还会随着堆叠层（即网络深度）的增多而进一步丰富。最新的研究证据[41, 44]表明，网络深度具有至关重要的意义，在难度极高的ImageNet数据集[36]上取得的最佳成果[41, 44, 13, 16]均采用了深度在16[41]到30[16]层之间的“极深”[41]模型。此外，许多其他复杂的视觉识别任务[8, 12, 7, 32, 27]也同

¹<http://image-net.org/challenges/LSVRC/2015/> 以及 <http://mscoco.org/dataset/#detections-challenge2015>。

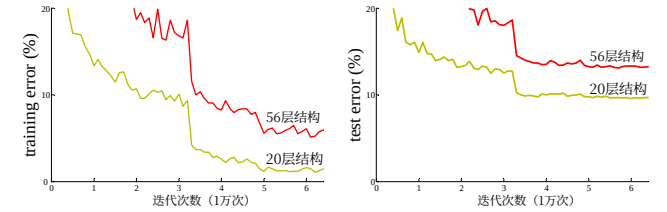


图1. 使用20层结构和56层结构的普通神经网络在CIFAR-10数据集上的训练误差（左侧）与测试误差（右侧）。层次越深的神经网络其训练误差越高，对应的测试误差也同样更高。ImageNet数据集上出现的类似现象可见于图4。

极大地受益于这类极深的模型。

鉴于深度的重要性，一个问题便随之产生：构建更优秀的神经网络是否仅仅意味着堆叠更多层结构？回答这个问题的一个障碍就是众所周知的梯度消失/爆炸问题[1, 9]，它从一开始就会阻碍神经网络的收敛。不过，通过归一化初始化[23, 9, 37, 13]以及中间归一化层[16]，这一问题已得到很大程度上解决，这些技术使得拥有数十层结构的神经网络在结合反向传播的随机梯度下降法（SGD）下也能开始收敛[22]。

当更深层的神经网络开始趋于收敛时，一个性能下降问题便显现出来：随着网络深度的增加，模型精度会先达到饱和和状态（这或许并不令人意外），随后便会迅速下降。出乎意料的是，这种性能下降并非由过拟合导致的；正如[11, 42]中所记载的，并且也通过我们的实验得到了充分验证，向已经足够深的模型中再增加更多层反而会导致更高的训练误差。图1展示了一个典型的实例。

训练精度的下降表明，并非所有模型都同样容易进行优化。让我们考虑一种层数较少的架构，以及在其基础上增加了更多层的更深层架构。对于这种更深层的模型，存在一种基于结构设计的解决方案：新增的层为恒等映射，而其他层则直接复制自己训练好的浅层模型。这种通过结构设计得出的解决方案的存在，意味着更深层的模型所产生的训练误差不应高于其浅层对应模型。但实验结果显示，我们目前所使用的求解器无法找到这样的解决方案。

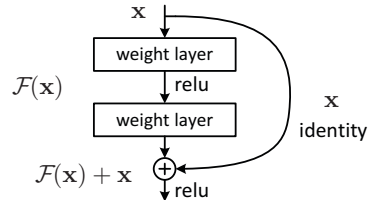


Figure 2. Residual learning: a building block.

are comparably good or better than the constructed solution (or unable to do so in feasible time).

In this paper, we address the degradation problem by introducing a *deep residual learning* framework. Instead of hoping each few stacked layers directly fit a desired underlying mapping, we explicitly let these layers fit a residual mapping. Formally, denoting the desired underlying mapping as $\mathcal{H}(\mathbf{x})$, we let the stacked nonlinear layers fit another mapping of $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$. The original mapping is recast into $\mathcal{F}(\mathbf{x}) + \mathbf{x}$. We hypothesize that it is easier to optimize the residual mapping than to optimize the original, unreferenced mapping. To the extreme, if an identity mapping were optimal, it would be easier to push the residual to zero than to fit an identity mapping by a stack of nonlinear layers.

The formulation of $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ can be realized by feedforward neural networks with “shortcut connections” (Fig. 2). Shortcut connections [2, 34, 49] are those skipping one or more layers. In our case, the shortcut connections simply perform *identity* mapping, and their outputs are added to the outputs of the stacked layers (Fig. 2). Identity shortcut connections add neither extra parameter nor computational complexity. The entire network can still be trained end-to-end by SGD with backpropagation, and can be easily implemented using common libraries (*e.g.*, Caffe [19]) without modifying the solvers.

We present comprehensive experiments on ImageNet [36] to show the degradation problem and evaluate our method. We show that: 1) Our extremely deep residual nets are easy to optimize, but the counterpart “plain” nets (that simply stack layers) exhibit higher training error when the depth increases; 2) Our deep residual nets can easily enjoy accuracy gains from greatly increased depth, producing results substantially better than previous networks.

Similar phenomena are also shown on the CIFAR-10 set [20], suggesting that the optimization difficulties and the effects of our method are not just akin to a particular dataset. We present successfully trained models on this dataset with over 100 layers, and explore models with over 1000 layers.

On the ImageNet classification dataset [36], we obtain excellent results by extremely deep residual nets. Our 152-layer residual net is the deepest network ever presented on ImageNet, while still having lower complexity than VGG nets [41]. Our ensemble has **3.57%** top-5 error on the

ImageNet *test* set, and *won the 1st place in the ILSVRC 2015 classification competition*. The extremely deep representations also have excellent generalization performance on other recognition tasks, and lead us to further *win the 1st places on: ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation* in ILSVRC & COCO 2015 competitions. This strong evidence shows that the residual learning principle is generic, and we expect that it is applicable in other vision and non-vision problems.

2. Related Work

Residual Representations. In image recognition, VLAD [18] is a representation that encodes by the residual vectors with respect to a dictionary, and Fisher Vector [30] can be formulated as a probabilistic version [18] of VLAD. Both of them are powerful shallow representations for image retrieval and classification [4, 48]. For vector quantization, encoding residual vectors [17] is shown to be more effective than encoding original vectors.

In low-level vision and computer graphics, for solving Partial Differential Equations (PDEs), the widely used Multigrid method [3] reformulates the system as subproblems at multiple scales, where each subproblem is responsible for the residual solution between a coarser and a finer scale. An alternative to Multigrid is hierarchical basis preconditioning [45, 46], which relies on variables that represent residual vectors between two scales. It has been shown [3, 45, 46] that these solvers converge much faster than standard solvers that are unaware of the residual nature of the solutions. These methods suggest that a good reformulation or preconditioning can simplify the optimization.

Shortcut Connections. Practices and theories that lead to shortcut connections [2, 34, 49] have been studied for a long time. An early practice of training multi-layer perceptrons (MLPs) is to add a linear layer connected from the network input to the output [34, 49]. In [44, 24], a few intermediate layers are directly connected to auxiliary classifiers for addressing vanishing/exploding gradients. The papers of [39, 38, 31, 47] propose methods for centering layer responses, gradients, and propagated errors, implemented by shortcut connections. In [44], an “inception” layer is composed of a shortcut branch and a few deeper branches.

Concurrent with our work, “highway networks” [42, 43] present shortcut connections with gating functions [15]. These gates are data-dependent and have parameters, in contrast to our identity shortcuts that are parameter-free. When a gated shortcut is “closed” (approaching zero), the layers in highway networks represent *non-residual* functions. On the contrary, our formulation always learns residual functions; our identity shortcuts are never closed, and all information is always passed through, with additional residual functions to be learned. In addition, high-

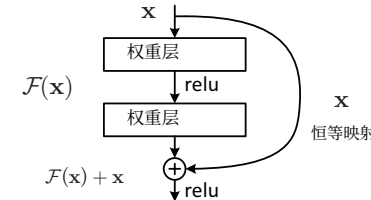


图2. 残差学习：其基本构成单元。

其性能与构建的解决方案相当或更优（或在可行时间内无法达到该水平）。

在本文中，我们通过引入一种深度残差学习框架来解决性能退化问题。我们并未试图让由若干堆叠层构成的结构直接适配某个预定的底层映射，而是明确要求这些层去适配一个残差映射。具体而言，将预定的底层映射表示为 $\mathcal{H}(\mathbf{x})$ ，则让这些堆叠的非线性层去适配另一个映射 $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$ 。原有的映射则被重新表示为 $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ 。我们假设优化残差映射要比优化原始的、没有参照依据的映射更为容易。极端情况下，如果恒等映射才是最优解，那么将残差值迫近于零会比通过非线性层堆叠来构建恒等映射更为简单。

$\mathcal{F}(\mathbf{x}) + \mathbf{x}$ 这一表达式可以通过带有“快捷连接”的前向神经网络来实现（见图2）。所谓快捷连接 [2, 34, 49]，指的是跳过一层或多层的连接方式。在本文的研究中，这类快捷连接仅执行恒等映射功能，其输出会被叠加到堆叠层的输出上（见图2）。由于采用了恒等映射的快捷连接，因此既不会增加额外的参数，也不会提升计算复杂度。整个网络依然可以通过结合随机梯度下降法与反向传播算法实现端到端的训练，而且只需使用常见的编程库（例如 Caffe [19]）即可轻松实现，无需对优化算法本身进行任何修改。

我们在ImageNet[36] 数据集上开展了大量实验，旨在展示性能退化问题并评估所提出的方法。实验结果表明：1) 我们所设计的深度极深的残差网络易于优化，而那些仅通过堆叠层构成的“普通”网络在层数增加时会出现更高的训练误差；2) 我们的深度残差网络即便层数大幅增加，也能获得显著的准确率提升，其表现远优于以往的各种神经网络。

在CIFAR-10数据集上也观察到了类似现象[20]，这表明我们所面临的优化难题以及该方法所带来的影响并非仅局限于某一特定数据集。我们展示了在该数据集上成功训练出的拥有超过100层结构的模型，同时还研究了层次数超过1000层的模型。

在ImageNet分类数据集 [36] 上，通过使用极为深度的深度残差网络，我们取得了优异的实验结果。我们所构建的152层深的残差网络是迄今为止在ImageNet上展示过的层次最深的神经网络，但其复杂度依然低于VGG网络 [41]。我们的集成模型在该数据集上的前5错误率为**3.57%**。

该模型在ImageNet 测试集上的表现极为出色，并在**2015年 ILSVRC**分类竞赛中夺得第一名。此外，这些极其深度的表示结构在其他识别任务中也展现出了优秀的泛化能力，助力我们在ILSVRC和COCO 2015竞赛中进一步在**ImageNet**检测、**ImageNet**定位、**COCO**检测以及**COCO**分割任务上均获得第一名。这些有力的证据表明残差学习原理具有通用性，我们预计它同样适用于其他视觉及非视觉领域的问题。

2. 相关工作

残差表示法。在图像识别领域，VLAD[18]是一种通过相对于字典的残差向量来对图像进行编码的表示方法，而费希尔向量 [30] 则可以被视作VLAD的概率化版本 [18]。这两种方法都是用于图像检索与分类 [4, 48]的强大浅层表示技术。在向量量化方面，研究显示对残差向量进行编码 [17] 比直接对原始向量进行编码更为高效。

在低级视觉和计算机图形学领域，为了解决偏微分方程（PDEs）的问题，人们广泛使用多网格方法 [3]，该方法会将原问题重新构建为多个不同尺度下的子问题，每个子问题负责处理较粗尺度与较细尺度之间的残差解。除了多网格方法之外，还有基于分层基预处理 [45, 46] 的替代方案，该方法依赖于能够表示两个尺度之间残差向量的特殊变量。已有研究表明[3, 45, 46]，这类求解器相较于那些没有考虑到解的残差特性的传统求解器，其收敛速度要快得多。这些方法都表明，合理的重新建模或预处理策略有助于简化优化过程。

快捷连接。长期以来，人们一直在研究那些与快捷连接相关的理论与实践方法 [2, 34, 49]。早期训练多层感知器（MLP）的一种做法是在神经网络的输入端与输出端之间添加一个线性层 [34, 49]。在 [44, 24] 中，为了应对梯度消失或梯度爆炸的问题，会直接将若干中间层与辅助分类器相连。[39, 38, 31, 47]的相关论文提出了通过快捷连接来调整层响应、梯度以及传播的误差的方法。而在 [44] 里，一种 “inception” 层则由一个快捷分支以及几个更深入的分支构成。

在与我们的研究同步进行的过程中，“高速公路网络架构” [42, 43]采用了带有门控函数的快捷连接 [15]。这些门控机制依赖于数据本身并包含相应参数，这与我们无需参数的恒等快捷路径截然不同。当带门控的快捷连接处于“关闭”状态（数值接近零）时，高速公路网络架构中的各层所表现出的就是非残差函数。相反，我们的建模方法始终会学习残差函数；我们的恒等快捷路径永远不会关闭，所有信息都会被原封不动地传递，同时还需要额外学习相应的残差函数。此外，这类高速公路网络架构在层深度极大增加的情况下——例如超过100层——并未展现出任何准确率提升。

way networks have not demonstrated accuracy gains with extremely increased depth (*e.g.*, over 100 layers).

3. Deep Residual Learning

3.1. Residual Learning

Let us consider $\mathcal{H}(\mathbf{x})$ as an underlying mapping to be fit by a few stacked layers (not necessarily the entire net), with $\mathbf{x}$ denoting the inputs to the first of these layers. If one hypothesizes that multiple nonlinear layers can asymptotically approximate complicated functions², then it is equivalent to hypothesize that they can asymptotically approximate the residual functions, *i.e.*, $\mathcal{H}(\mathbf{x}) - \mathbf{x}$ (assuming that the input and output are of the same dimensions). So rather than expect stacked layers to approximate $\mathcal{H}(\mathbf{x})$, we explicitly let these layers approximate a residual function $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$. The original function thus becomes $\mathcal{F}(\mathbf{x}) + \mathbf{x}$. Although both forms should be able to asymptotically approximate the desired functions (as hypothesized), the ease of learning might be different.

This reformulation is motivated by the counterintuitive phenomena about the degradation problem (Fig. 1, left). As we discussed in the introduction, if the added layers can be constructed as identity mappings, a deeper model should have training error no greater than its shallower counterpart. The degradation problem suggests that the solvers might have difficulties in approximating identity mappings by multiple nonlinear layers. With the residual learning reformulation, if identity mappings are optimal, the solvers may simply drive the weights of the multiple nonlinear layers toward zero to approach identity mappings.

In real cases, it is unlikely that identity mappings are optimal, but our reformulation may help to precondition the problem. If the optimal function is closer to an identity mapping than to a zero mapping, it should be easier for the solver to find the perturbations with reference to an identity mapping, than to learn the function as a new one. We show by experiments (Fig. 7) that the learned residual functions in general have small responses, suggesting that identity mappings provide reasonable preconditioning.

3.2. Identity Mapping by Shortcuts

We adopt residual learning to every few stacked layers. A building block is shown in Fig. 2. Formally, in this paper we consider a building block defined as:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}. \quad (1)$$

Here $\mathbf{x}$ and $\mathbf{y}$ are the input and output vectors of the layers considered. The function $\mathcal{F}(\mathbf{x}, \{W_i\})$ represents the residual mapping to be learned. For the example in Fig. 2 that has two layers, $\mathcal{F} = W_2\sigma(W_1\mathbf{x})$ in which σ denotes

ReLU [29] and the biases are omitted for simplifying notations. The operation $\mathcal{F} + \mathbf{x}$ is performed by a shortcut connection and element-wise addition. We adopt the second nonlinearity after the addition (*i.e.*, $\sigma(\mathbf{y})$, see Fig. 2).

The shortcut connections in Eqn.(1) introduce neither extra parameter nor computation complexity. This is not only attractive in practice but also important in our comparisons between plain and residual networks. We can fairly compare plain/residual networks that simultaneously have the same number of parameters, depth, width, and computational cost (except for the negligible element-wise addition).

The dimensions of $\mathbf{x}$ and $\mathcal{F}$ must be equal in Eqn.(1). If this is not the case (*e.g.*, when changing the input/output channels), we can perform a linear projection W_s by the shortcut connections to match the dimensions:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x}. \quad (2)$$

We can also use a square matrix W_s in Eqn.(1). But we will show by experiments that the identity mapping is sufficient for addressing the degradation problem and is economical, and thus W_s is only used when matching dimensions.

The form of the residual function $\mathcal{F}$ is flexible. Experiments in this paper involve a function $\mathcal{F}$ that has two or three layers (Fig. 5), while more layers are possible. But if $\mathcal{F}$ has only a single layer, Eqn.(1) is similar to a linear layer: $\mathbf{y} = W_1\mathbf{x} + \mathbf{x}$, for which we have not observed advantages.

We also note that although the above notations are about fully-connected layers for simplicity, they are applicable to convolutional layers. The function $\mathcal{F}(\mathbf{x}, \{W_i\})$ can represent multiple convolutional layers. The element-wise addition is performed on two feature maps, channel by channel.

3.3. Network Architectures

We have tested various plain/residual nets, and have observed consistent phenomena. To provide instances for discussion, we describe two models for ImageNet as follows.

Plain Network. Our plain baselines (Fig. 3, middle) are mainly inspired by the philosophy of VGG nets [41] (Fig. 3, left). The convolutional layers mostly have 3×3 filters and follow two simple design rules: (i) for the same output feature map size, the layers have the same number of filters; and (ii) if the feature map size is halved, the number of filters is doubled so as to preserve the time complexity per layer. We perform downsampling directly by convolutional layers that have a stride of 2. The network ends with a global average pooling layer and a 1000-way fully-connected layer with softmax. The total number of weighted layers is 34 in Fig. 3 (middle).

It is worth noticing that our model has *fewer* filters and *lower* complexity than VGG nets [41] (Fig. 3, left). Our 34-layer baseline has 3.6 billion FLOPs (multiply-adds), which is only 18% of VGG-19 (19.6 billion FLOPs).

这类高速公路网络架构在层深度极大增加的情况下——例如超过100层——并未展现出任何准确率提升。

3. 深度残差学习

3.1. 残差学习

让我们将 $\mathcal{H}(\mathbf{x})$ 视为一个需要通过若干堆叠层来拟合的底层映射（不一定是整个网络），其中 $\mathbf{x}$ 表示这些层中第一层的输入。如果假设多个非线性层能够渐近地逼近复杂的函数²，那么这就等同于假设它们能够渐近地逼近残差函数，也就是说， $\mathcal{H}(\mathbf{x}) - \mathbf{x}$ （前提是输入与输出的维度相同）。因此，我们并非期望堆叠层去逼近 $\mathcal{H}(\mathbf{x})$ ，而是明确让这些层去逼近某个残差函数 $\mathcal{F}(\mathbf{x})$ ，即 $\mathcal{H}(\mathbf{x}) - \mathbf{x}$ 。这样一来，原始函数就变成了 $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ 。尽管按照假设，这两种形式都应该能够渐近地逼近目标函数，但它们的学习难度可能会有所不同。

这种重新表述的出发点，是源于关于性能退化问题的一些反直觉现象（见图1左侧）。正如我们在引言中讨论的，如果新增的层可以被构造为恒等映射，那么结构更深的模型其训练误差应当不会高于结构较浅的模型。然而性能退化问题表明，优化器在利用多个非线性层来逼近恒等映射时可能会遇到困难。通过采用残差学习的重新表述方式，若恒等映射是最佳选择，优化器只需将多个非线性层的权重逐步趋近于零，即可实现对恒等映射的逼近。

在实际应用中，恒等映射往往并非最优选择，但我们的重新建模方法有助于为该问题提供良好的预处理条件。如果最优函数的形态更接近恒等映射而非零映射，那么参考恒等映射来寻找扰动项，会比将其视为全新的函数来学习要更为容易。实验结果（见图7）表明，所学习的残差函数通常响应较小，这一现象说明恒等映射确实能够提供有效的预处理效果。

3.2. 通过快捷连接实现恒等映射

我们在每隔几层堆叠层之后应用残差学习技术。图2展示了一个这样的构建模块。严格来说，本文中所探讨的构建模块定义如下：

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}. \quad (1)$$

此处， $\mathbf{x}$ 与 $\mathbf{y}$ 分别代表所分析各层的输入向量与输出向量。函数 $\mathcal{F}(\mathbf{x}, \{W_i\})$ 表示需要被学习的残差映射关系。以图2中包含两层的示例而言， $\mathcal{F} = W_2\sigma(W_1\mathbf{x})$ 中的 σ 即表示该表达式。

为简化表达，文中未列出ReLU [29]及各种偏置项。运算 $\mathcal{F} + \mathbf{x}$ 是通过快捷连接与逐元素相加来实现的。在完成相加之后，我们会引入第二种非线性运算（即， $\sigma(\mathbf{y})$ ），详见图2。

方程(1)中的快捷连接既不会引入额外的参数，也不会增加计算复杂度。这一特点不仅在实践中具有很大优势，对于我们比较普通神经网络与残差神经网络而言也至关重要。这样一来，我们便可以公平地对比那些在参数数量、深度、宽度以及计算成本方面完全相同的普通神经网络与残差神经网络（仅元素级加法这一项可忽略不计）。

在方程(1)中， $\mathbf{x}$ 与 $\mathcal{F}$ 的维度必须保持一致。如果出现不一致的情况（例如在更改输入/输出通道时），我们可以通过快捷连接进行线性投影 W_s ，从而使得二者维度相匹配。

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x}. \quad (2)$$

我们也可以在方程(1)中使用方阵 W_s 。但实验结果表明，恒等映射本身就足以解决性能退化问题，且更为经济高效；因此 W_s 仅用于处理维度匹配的场景。

残差函数 $\mathcal{F}$ 的形式具有灵活性。本文中的实验所使用的函数 $\mathcal{F}$ 通常包含两到三层结构（见图5），当然也可以使用更多层。不过如果 $\mathcal{F}$ 仅包含单层，那么方程(1)实际上就相当于一个全连接层： $\mathbf{y} = W_1\mathbf{x} + \mathbf{x}$ ，而这种结构并未展现出任何优势。

需要说明的是，虽然上述表述为简化起见以全连接层为例，但实际上它们同样适用于卷积层。函数 $\mathcal{F}(\mathbf{x}, \{W_i\})$ 能够表示多个卷积层，而元素级加法则针对两个特征图的各个通道逐个进行的。

3.3. 神经网络架构

我们测试了多种普通神经网络与残差网络，发现其中存在一些一致的现象。为便于讨论，下文将详细介绍两种用于ImageNet数据集的模型。

普通神经网络。我们所使用的普通基准模型（图3中间部分）主要借鉴了VGG网络 [41]（图3左侧）的设计理念。这些卷积层通常配备有 3×3 个滤波器，并遵循两条简单的设计原则：其一，当输出特征图尺寸相同时，各层的滤波器数量保持一致；其二，若特征图尺寸减半，则滤波器数量需翻倍，以此维持每层的时间复杂度不变。我们通过步长为2的卷积层来实现下采样操作。该神经网络的最后部分包含一个全局平均池化层以及一个带有softmax函数的1000维全连接层。如图3中间部分所示，这类模型的加权层总数为34层。

值得注意的是，与VGG网络 [41]（见图3左侧）相比，我们的模型所拥有的滤波器数量更少，且复杂度也更低。我们这个拥有34层的基准模型所需的浮点运算次数为36亿次乘加运算，这一数值仅相当于VGG-19的18%（VGG-19的浮点运算次数为196亿次）。

²This hypothesis, however, is still an open question. See [28].

²不过，这一假设目前仍属于未解之谜，详情可参见 [28]。

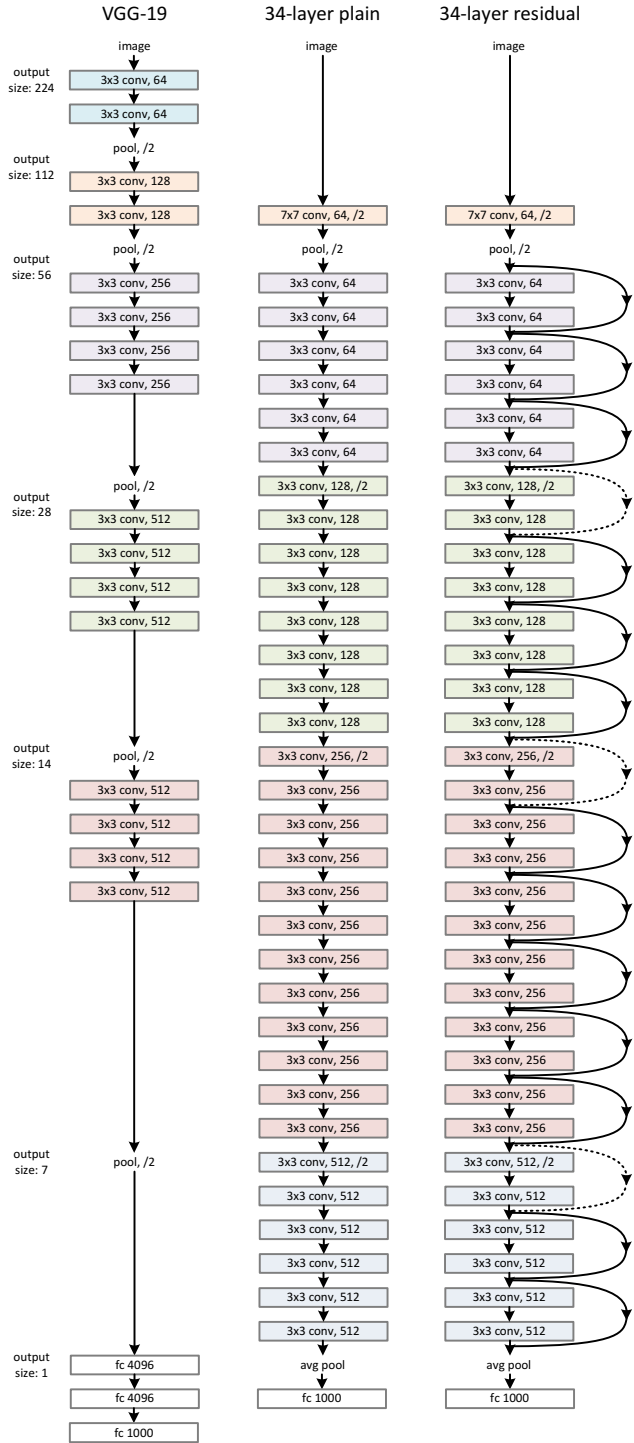


Figure 3. Example network architectures for ImageNet. **Left:** the VGG-19 model [41] (19.6 billion FLOPs) as a reference. **Middle:** a plain network with 34 parameter layers (3.6 billion FLOPs). **Right:** a residual network with 34 parameter layers (3.6 billion FLOPs). The dotted shortcuts increase dimensions. **Table 1** shows more details and other variants.

Residual Network. Based on the above plain network, we insert shortcut connections (Fig. 3, right) which turn the network into its counterpart residual version. The identity shortcuts (Eqn.(1)) can be directly used when the input and output are of the same dimensions (solid line shortcuts in Fig. 3). When the dimensions increase (dotted line shortcuts in Fig. 3), we consider two options: (A) The shortcut still performs identity mapping, with extra zero entries padded for increasing dimensions. This option introduces no extra parameter; (B) The projection shortcut in Eqn.(2) is used to match dimensions (done by 1×1 convolutions). For both options, when the shortcuts go across feature maps of two sizes, they are performed with a stride of 2.

3.4. Implementation

Our implementation for ImageNet follows the practice in [21, 41]. The image is resized with its shorter side randomly sampled in [256, 480] for scale augmentation [41]. A 224×224 crop is randomly sampled from an image or its horizontal flip, with the per-pixel mean subtracted [21]. The standard color augmentation in [21] is used. We adopt batch normalization (BN) [16] right after each convolution and before activation, following [16]. We initialize the weights as in [13] and train all plain/residual nets from scratch. We use SGD with a mini-batch size of 256. The learning rate starts from 0.1 and is divided by 10 when the error plateaus, and the models are trained for up to 60×10^4 iterations. We use a weight decay of 0.0001 and a momentum of 0.9. We do not use dropout [14], following the practice in [16].

In testing, for comparison studies we adopt the standard 10-crop testing [21]. For best results, we adopt the fully-convolutional form as in [41, 13], and average the scores at multiple scales (images are resized such that the shorter side is in $\{224, 256, 384, 480, 640\}$).

4. Experiments

4.1. ImageNet Classification

We evaluate our method on the ImageNet 2012 classification dataset [36] that consists of 1000 classes. The models are trained on the 1.28 million training images, and evaluated on the 50k validation images. We also obtain a final result on the 100k test images, reported by the test server. We evaluate both top-1 and top-5 error rates.

Plain Networks. We first evaluate 18-layer and 34-layer plain nets. The 34-layer plain net is in Fig. 3 (middle). The 18-layer plain net is of a similar form. See Table 1 for detailed architectures.

The results in Table 2 show that the deeper 34-layer plain net has higher validation error than the shallower 18-layer plain net. To reveal the reasons, in Fig. 4 (left) we compare their training/validation errors during the training procedure. We have observed the degradation problem - the

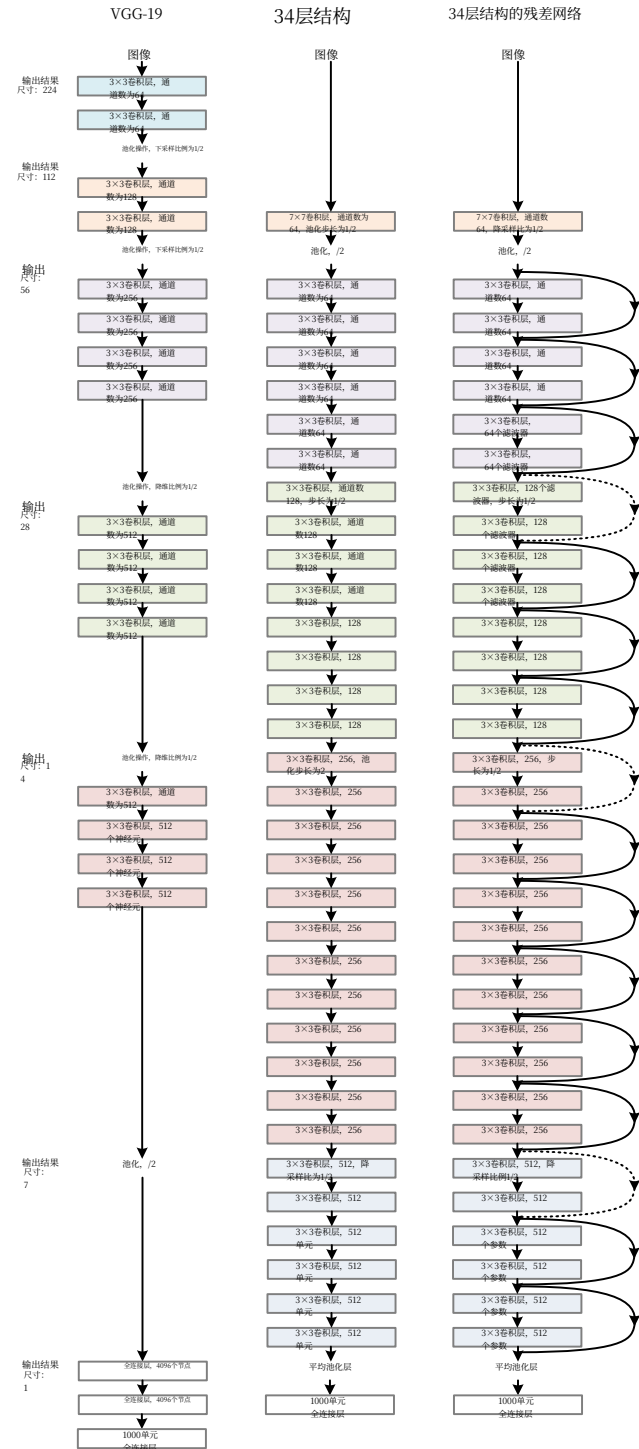


图3. ImageNet数据集的典型神经网络架构。左侧所示为作为参考的VGG-19模型 [41]，其浮点运算次数达196亿次；中间为仅包含34层参数的普通神经网络，浮点运算次数为36亿次；右侧则为同样具有34层参数的残差网络，浮点运算次数同样为36亿次。图中虚线所示的路径用于提升特征图的维度。表1列出了更多细节及其他变体设计。

残差网络。在上述普通神经网络的基础上，我们加入了快捷连接（见图3右侧），从而将该网络转化为对应的残差版本。当输入与输出的维度相同时，可直接使用恒等快捷路径（即方程(1)中所描述的那种，见图3中的实线快捷路径）。而当维度发生变化时（见图3中的虚线快捷路径），我们有两种处理方式：(A) 快捷路径依然执行恒等映射，同时为适配更高的维度而添加额外的零元素；这种方式不会引入任何新的参数；(B) 则采用方程(2)中描述的投影快捷路径来匹配不同维度，该操作可通过 1×1 次卷积来实现。对于这两种方式而言，若快捷路径需要跨越尺寸不同的特征图，其步长都将设置为2。

3.4. 实现方式

我们在处理ImageNet数据集时采用了 [21, 41]中的常用方法。首先会随机选取图像较短的一边对图像进行尺寸调整 [256, 480]，以此实现尺度增强 [41]。随后会从原图像或其水平翻转后的版本中随机选取一个 224×224 大小的区域，再从中减去每个像素的均值 [21]。同时还会应用 [21]中规定的标准颜色增强技术。我们遵循 [16]的建议，在每次卷积运算之后、激活函数之前加入批归一化 (BN) [16]。权重初始化方式与 [13] 中的描述一致，所有普通神经网络和残差网络都会从零开始训练。训练过程中采用的优化算法为随机梯度下降法 (SGD)，小批量大小设置为256。学习率初始值为0.1，当误差不再下降时每10倍降低一次，模型的训练迭代次数最多可达 60×10^4 次。此外我们还设置了0.0001的权重衰减系数和0.9的动量值。按照 [16]中的做法，我们并未使用丢弃机制 [14]。

在测试过程中，为便于对比研究，我们采用了标准的10张图像裁剪测试方法 [21]。为获得最佳结果，我们采用与 [41, 13]，中相同的全卷积结构，并对多个不同尺寸下的得分取平均值（图像会被重新调整大小，使其短边长度符合 $\{224, 256, 384, 480, 640\}$ 的要求）。

4. 实验

4.1. ImageNet分类

我们在包含1000个类别的ImageNet 2012分类数据集 [36]上对我们的方法进行了评估。这些模型在128万张训练图像上进行训练，然后在5万张验证图像上进行性能测试。此外，我们还通过测试服务器得到了在10万张测试图像上的最终结果。我们会同时统计前1错误率和前5错误率。

普通网络。我们首先对18层结构和34层结构的普通网络进行评估。其中34层结构的普通网络展示在图3的中间位置，18层结构的普通网络形式与之类似，详细的架构信息请参见表1。

表2中的结果显示，深度更高的34层结构普通网络的验证误差要高于深度较低的18层结构普通网络。为探寻其中原因，图4（左侧）展示了训练过程中这两种网络的训练误差与验证误差的对比情况。我们观察到了性能退化问题——

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
		3×3 max pool, stride 2				
conv2_x	56×56	$\begin{bmatrix} 3\times 3, 64 \\ 3\times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 64 \\ 3\times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3\times 3, 128 \\ 3\times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 128 \\ 3\times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3\times 3, 256 \\ 3\times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 256 \\ 3\times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3\times 3, 512 \\ 3\times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 512 \\ 3\times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Table 1. Architectures for ImageNet. Building blocks are shown in brackets (see also Fig. 5), with the numbers of blocks stacked. Down-sampling is performed by conv3_1, conv4_1, and conv5_1 with a stride of 2.

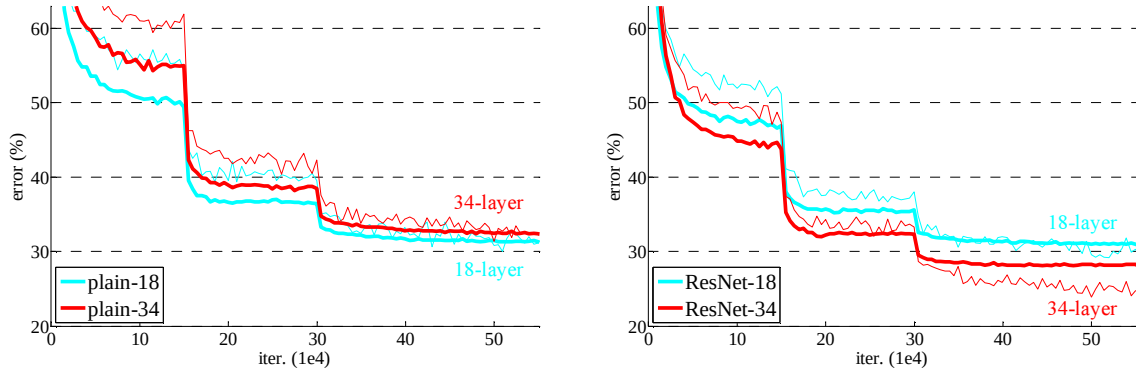


Figure 4. Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	25.03

Table 2. Top-1 error (% , 10-crop testing) on ImageNet validation. Here the ResNets have no extra parameter compared to their plain counterparts. Fig. 4 shows the training procedures.

34-layer plain net has higher *training* error throughout the whole training procedure, even though the solution space of the 18-layer plain network is a subspace of that of the 34-layer one.

We argue that this optimization difficulty is *unlikely* to be caused by vanishing gradients. These plain networks are trained with BN [16], which ensures forward propagated signals to have non-zero variances. We also verify that the backward propagated gradients exhibit healthy norms with BN. So neither forward nor backward signals vanish. In fact, the 34-layer plain net is still able to achieve competitive accuracy (Table 3), suggesting that the solver works to some extent. We conjecture that the deep plain nets may have exponentially low convergence rates, which impact the

reducing of the training error³. The reason for such optimization difficulties will be studied in the future.

Residual Networks. Next we evaluate 18-layer and 34-layer residual nets (*ResNets*). The baseline architectures are the same as the above plain nets, expect that a shortcut connection is added to each pair of 3×3 filters as in Fig. 3 (right). In the first comparison (Table 2 and Fig. 4 right), we use identity mapping for all shortcuts and zero-padding for increasing dimensions (option A). So they have *no extra parameter* compared to the plain counterparts.

We have three major observations from Table 2 and Fig. 4. First, the situation is reversed with residual learning – the 34-layer ResNet is better than the 18-layer ResNet (by 2.8%). More importantly, the 34-layer ResNet exhibits considerably lower training error and is generalizable to the validation data. This indicates that the degradation problem is well addressed in this setting and we manage to obtain accuracy gains from increased depth.

Second, compared to its plain counterpart, the 34-layer

³We have experimented with more training iterations ($3\times$) and still observed the degradation problem, suggesting that this problem cannot be feasibly addressed by simply using more iterations.

层数参数n	输出尺寸	18层结构	34层结构	50层结构	101层结构	152层结构
conv1	112×112	7×7, 64, 步长为2				
池		3×3最大值池化, 步长为2				
conv2x层	56×56	$\begin{bmatrix} 3\times 3, 64 \\ 3\times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 64 \\ 3\times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 64 \\ 3\times 3, 64 \\ 1\times 1, 256 \end{bmatrix} \times 3$
conv3x层	28×28	$\begin{bmatrix} 3\times 3, 128 \\ 3\times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 128 \\ 3\times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1\times 1, 128 \\ 3\times 3, 128 \\ 1\times 1, 512 \end{bmatrix} \times 8$
conv4x层	14×14	$\begin{bmatrix} 3\times 3, 256 \\ 3\times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 256 \\ 3\times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1\times 1, 256 \\ 3\times 3, 256 \\ 1\times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3\times 3, 512 \\ 3\times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3\times 3, 512 \\ 3\times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1\times 1, 512 \\ 3\times 3, 512 \\ 1\times 1, 2048 \end{bmatrix} \times 3$
	1×1	平均池化层、1000维全连接层以及softmax函数				
浮点运算次数		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

表1. ImageNet数据集对应的架构设计。各构建模块以括号形式标出（亦可参考图5），同时标注了这些模块的堆叠数量；下采样操作是通过步长为2的conv3_1层、conv4_1层以及conv5_1层来实现的。

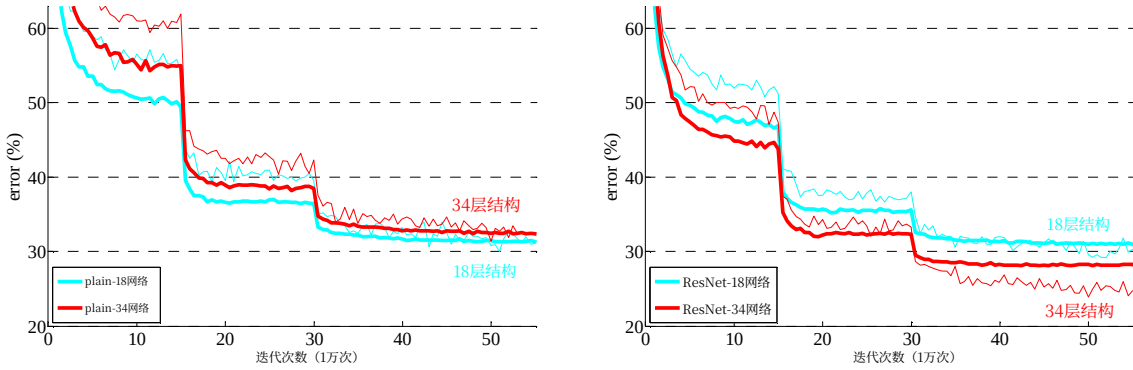


图4. 在**ImageNet**数据集上的训练情况。细曲线代表中心裁剪区域的训练误差，粗曲线则代表同一区域的验证误差。左侧为具有18层结构和34层结构的普通网络，右侧为具有18层结构和34层结构的ResNet系列模型。在该图中，这类残差神经网络与相应的普通网络相比并没有额外的参数。

	普通网络	ResNet网络
18层结构	27.94	27.88
34层结构	28.54	25.03

表2. ImageNet验证集上的前1错误率（%，10次裁剪测试）。此处，ResNet系列模型与对应的普通版本相比没有额外的层次参数n。图4展示了训练流程。

即便18层结构普通网络的解空间是34层结构普通网络解空间的一个子空间，后者在整个训练过程中的训练误差依然更高。

我们认为，这种优化难题不太可能是由梯度消失现象引起的。这些普通网络在训练时会采用批归一化 [16]，该技术能够确保前向传播信号的方差不为零。此外我们还验证了，在应用批归一化后，后向传播梯度的范数处于正常范围，因此无论是前向还是后向的信号都不会消失。事实上，即便是采用34层结构的普通网络，依然能够达到具有竞争力的准确率（见表3），这表明优化算法在某种程度上是有效的。我们推测，深层普通网络的收敛速率可能会低至指数级，而这恰恰会影响训练误差的降低。至于造成此类优化难题的具体原因，未来还将进一步研究。

这类优化难题的原因将在日后进行深入探讨。

残差网络。接下来我们将对18层结构以及34层结构的残差网络（*ResNets*）进行评估。这些基准架构与上述普通网络相同，不同之处在于如图3（右侧）所示，每对 3×3 滤波器之间都添加了快捷连接。在第一次对比中（表2及图4右侧），我们对所有快捷连接采用恒等映射处理，而对于维度提升则使用零填充（选项A）。因此与对应的普通网络相比，这类网络没有额外的参数。

从表2和图4中我们可以得出三个主要观察结果。首先，采用残差学习后情况发生了逆转——34层结构的ResNet网络的表现优于18层结构的ResNet网络（优势达2.8%）。更重要的是，34层结构的ResNet网络具有显著更低的训练误差，并且能够很好地泛化到验证数据上。这表明在该实验环境中性能退化问题已得到有效解决，而且通过增加网络层数我们确实实现了准确率提升。

其次，与普通的对应模型相比，具备34层结构的ResNet网络

³我们尝试过增加火车的运行次数（ $3\times$ ），但依然出现了性能退化问题，这表明仅通过提升运行次数是无法有效解决该问题的。

model	top-1 err.	top-5 err.
VGG-16 [41]	28.07	9.33
GoogLeNet [44]	-	9.15
PReLU-net [13]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

Table 3. Error rates (% , **10-crop** testing) on ImageNet validation. VGG-16 is based on our test. ResNet-50/101/152 are of option B that only uses projections for increasing dimensions.

method	top-1 err.	top-5 err.
VGG [41] (ILSVRC’14)	-	8.43 [†]
GoogLeNet [44] (ILSVRC’14)	-	7.89
VGG [41] (v5)	24.4	7.1
PReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

Table 4. Error rates (%) of **single-model** results on the ImageNet validation set (except [†] reported on the test set).

method	top-5 err. (test)
VGG [41] (ILSVRC’14)	7.32
GoogLeNet [44] (ILSVRC’14)	6.66
VGG [41] (v5)	6.8
PReLU-net [13]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC’15)	3.57

Table 5. Error rates (%) of **ensembles**. The top-5 error is on the test set of ImageNet and reported by the test server.

ResNet reduces the top-1 error by 3.5% (Table 2), resulting from the successfully reduced training error (Fig. 4 right *vs.* left). This comparison verifies the effectiveness of residual learning on extremely deep systems.

Last, we also note that the 18-layer plain/residual nets are comparably accurate (Table 2), but the 18-layer ResNet converges faster (Fig. 4 right *vs.* left). When the net is “not overly deep” (18 layers here), the current SGD solver is still able to find good solutions to the plain net. In this case, the ResNet eases the optimization by providing faster convergence at the early stage.

Identity *vs.* Projection Shortcuts. We have shown that

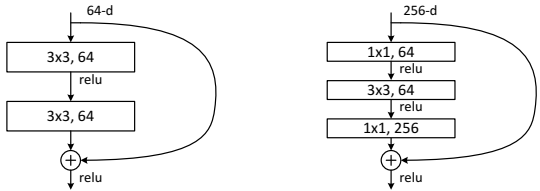


Figure 5. A deeper residual function $\mathcal{F}$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 3 for ResNet-34. Right: a “bottleneck” building block for ResNet-50/101/152.

parameter-free, identity shortcuts help with training. Next we investigate projection shortcuts (Eqn.(2)). In Table 3 we compare three options: (A) zero-padding shortcuts are used for increasing dimensions, and all shortcuts are parameter-free (the same as Table 2 and Fig. 4 right); (B) projection shortcuts are used for increasing dimensions, and other shortcuts are identity; and (C) all shortcuts are projections.

Table 3 shows that all three options are considerably better than the plain counterpart. B is slightly better than A. We argue that this is because the zero-padded dimensions in A indeed have no residual learning. C is marginally better than B, and we attribute this to the extra parameters introduced by many (thirteen) projection shortcuts. But the small differences among A/B/C indicate that projection shortcuts are not essential for addressing the degradation problem. So we do not use option C in the rest of this paper, to reduce memory/time complexity and model sizes. Identity shortcuts are particularly important for not increasing the complexity of the bottleneck architectures that are introduced below.

Deeper Bottleneck Architectures. Next we describe our deeper nets for ImageNet. Because of concerns on the training time that we can afford, we modify the building block as a *bottleneck* design⁴. For each residual function $\mathcal{F}$, we use a stack of 3 layers instead of 2 (Fig. 5). The three layers are 1×1 , 3×3 , and 1×1 convolutions, where the 1×1 layers are responsible for reducing and then increasing (restoring) dimensions, leaving the 3×3 layer a bottleneck with smaller input/output dimensions. Fig. 5 shows an example, where both designs have similar time complexity.

The parameter-free identity shortcuts are particularly important for the bottleneck architectures. If the identity shortcut in Fig. 5 (right) is replaced with projection, one can show that the time complexity and model size are doubled, as the shortcut is connected to the two high-dimensional ends. So identity shortcuts lead to more efficient models for the bottleneck designs.

50-layer ResNet: We replace each 2-layer block in the

⁴Deeper *non*-bottleneck ResNets (*e.g.*, Fig. 5 left) also gain accuracy from increased depth (as shown on CIFAR-10), but are not as economical as the bottleneck ResNets. So the usage of bottleneck designs is mainly due to practical considerations. We further note that the degradation problem of plain nets is also witnessed for the bottleneck designs.

模型	Top-1错误率	Top-5错误率
VGG-16 [41]	28.07	9.33
GoogLeNet [44]	-	9.15
PReLU-net [13]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C版本	24.19	7.40
ResNet-50网络	22.85	6.71
ResNet-101网络	21.75	6.05
ResNet-152网络	21.43	5.71

表3. ImageNet验证集上的误差率（单位：%，**10裁剪**测试方式）。VGG-16的数值基于我们的测试结果；而ResNet-50/101/152属于选项B，这类模型仅通过投影操作来实现特征维度提升。

方法	Top-1错误率	Top-5错误率
VGG [41] (ILSVRC 2014竞赛)	-	8.43 [†]
GoogLeNet [44] (ILSVRC 2014竞赛)	-	7.89
VGG [41] (v5)	24.4	7.1
PReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B版本	21.84	5.71
ResNet-34 C版本	21.53	5.60
ResNet-50网络	20.74	5.25
ResNet-101网络	19.87	4.60
ResNet-152网络	19.38	4.49

表4. ImageNet验证集上**单模型**方法的误差率（%）（测试集上的数据除外，详见[†]）。

方法	Top-5错误率（ 测试集数据 ）
VGG [41] (ILSVRC 2014竞赛)	7.32
GoogLeNet算法 [44] (ILSVRC 2014竞赛)	6.66
VGG [41] (v5)	6.8
PReLU-net [13]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC’15竞赛)	3.57

表5: **集成模型**的误差率（%）。前5错误率是在ImageNet的测试集上测得的，并由测试服务器给出。

能够将前1错误率降低3.5%（见表2），这一成效得益于训练误差的成功控制（见图4右侧与左侧）。这一对比充分证明了残差学习在处理极深层系统时的有效性。

最后需要指出的是，18层结构的普通网络与残差网络具有相近的准确度（见表2），但18层结构的ResNet收敛速度更快（见图4右侧与左侧对比）。当网络深度“并非过深”（此处为18层）时，现有的SGD优化器依然能够为普通网络找到良好的优化解。而在这种情形下，ResNet由于能在早期阶段实现更快的收敛，从而简化了优化过程。

恒等映射 与。投影快捷路径。我们已经证明

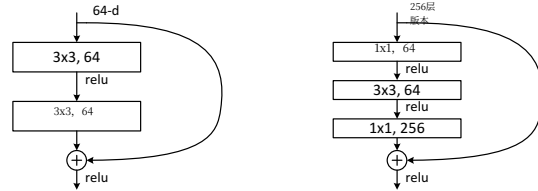


图5. 用于ImageNet的更深层次的残差函数 $\mathcal{F}$ 。左侧为ResNet-34所采用的、基于 56×56 特征图的构建模块，如图3所示；右侧则为ResNet-50/101/152所使用的“瓶颈”型构建模块。

无需参数的恒等快捷路径有助于训练过程。接下来我们将研究投影快捷路径（公式(2)）。在表3中，我们对比了三种方案：(A) 使用零填充快捷路径来增加维度，且所有快捷路径均无需参数（这与表2及右侧的图4相同）；(B) 使用投影快捷路径来增加维度，而其他快捷路径则为恒等映射；(C) 所有快捷路径均为投影操作。

表3显示，这三种方案的表现都远优于单纯的对应方案。其中B方案略好于A方案，我们认为这是因为A方案中经过零填充处理后的维度实际上并不涉及残差学习。C方案的表现略好于B方案，我们将其归因于众多（共十三个）投影快捷路径所带来的额外参数。不过A、B、C方案之间的差异较小，说明投影快捷路径并非解决性能退化问题的必要条件。因此，在本文的后续部分我们不再使用C方案，以此降低内存/时间复杂度以及模型规模。此外，恒等快捷路径对于避免增加下文将要介绍的瓶颈架构的复杂性尤为为重要。

更深入的瓶颈架构。接下来我们将介绍专为ImageNet设计的更深层次网络结构。考虑到可承受的训练时间限制，我们对构建模块进行了修改，采用了瓶颈设计⁴。对于每一种残差函数 $\mathcal{F}$ ，我们不再使用2层，而是采用3层堆叠的结构（见图5）。这三层分别是 1×1 、 3×3 和 1×1 卷积层，其中前两层负责先缩小再扩大（恢复）维度，从而使最后一层成为具有较小输入/输出尺寸的瓶颈层。图5展示了一个示例，两种设计的时间复杂度大致相当。

对于那些采用瓶颈架构的模型而言，无参数恒等式快捷路径尤为为重要。如果将图5（右侧）中的恒等快捷路径替换为投影操作，就可以发现由于该快捷路径与两个高维端相连，模型的时间复杂度与模型规模都会翻倍。正因如此，恒等快捷路径能够帮助这类瓶颈设计实现更高效的模型性能。

50层ResNet: 我们替换了其中的每一个2层块

⁴结构更深的非瓶颈ResNet系列模型（例如，图5左侧所示），虽然也能通过增加层数提升在CIFAR-10数据集上的准确率，但其效率却不及瓶颈ResNet网络。因此，选择瓶颈设计主要是出于各种实际考量因素。此外还需要注意的是，普通网络也会出现性能退化问题，瓶颈设计同样无法避免这一现象。

34-layer net with this 3-layer bottleneck block, resulting in a 50-layer ResNet (Table 1). We use option B for increasing dimensions. This model has 3.8 billion FLOPs.

101-layer and 152-layer ResNets: We construct 101-layer and 152-layer ResNets by using more 3-layer blocks (Table 1). Remarkably, although the depth is significantly increased, the 152-layer ResNet (11.3 billion FLOPs) still has *lower complexity* than VGG-16/19 nets (15.3/19.6 billion FLOPs).

The 50/101/152-layer ResNets are more accurate than the 34-layer ones by considerable margins (Table 3 and 4). We do not observe the degradation problem and thus enjoy significant accuracy gains from considerably increased depth. The benefits of depth are witnessed for all evaluation metrics (Table 3 and 4).

Comparisons with State-of-the-art Methods. In Table 4 we compare with the previous best single-model results. Our baseline 34-layer ResNets have achieved very competitive accuracy. Our 152-layer ResNet has a single-model top-5 validation error of 4.49%. This single-model result outperforms all previous ensemble results (Table 5). We combine six models of different depth to form an ensemble (only with two 152-layer ones at the time of submitting). This leads to **3.57%** top-5 error on the test set (Table 5). *This entry won the 1st place in ILSVRC 2015.*

4.2. CIFAR-10 and Analysis

We conducted more studies on the CIFAR-10 dataset [20], which consists of 50k training images and 10k testing images in 10 classes. We present experiments trained on the training set and evaluated on the test set. Our focus is on the behaviors of extremely deep networks, but not on pushing the state-of-the-art results, so we intentionally use simple architectures as follows.

The plain/residual architectures follow the form in Fig. 3 (middle/right). The network inputs are 32×32 images, with the per-pixel mean subtracted. The first layer is 3×3 convolutions. Then we use a stack of $6n$ layers with 3×3 convolutions on the feature maps of sizes $\{32, 16, 8\}$ respectively, with $2n$ layers for each feature map size. The numbers of filters are $\{16, 32, 64\}$ respectively. The subsampling is performed by convolutions with a stride of 2. The network ends with a global average pooling, a 10-way fully-connected layer, and softmax. There are totally $6n+2$ stacked weighted layers. The following table summarizes the architecture:

output map size	32×32	16×16	8×8
# layers	$1+2n$	$2n$	$2n$
# filters	16	32	64

When shortcut connections are used, they are connected to the pairs of 3×3 layers (totally $3n$ shortcuts). On this dataset we use identity shortcuts in all cases (*i.e.*, option A),

method			error (%)
Maxout [10]			9.38
NIN [25]			8.81
DSN [24]			8.22
	# layers	# params	
FitNet [35]	19	2.5M	8.39
Highway [42, 43]	19	2.3M	7.54 (7.72±0.16)
Highway [42, 43]	32	1.25M	8.80
ResNet	20	0.27M	8.75
ResNet	32	0.46M	7.51
ResNet	44	0.66M	7.17
ResNet	56	0.85M	6.97
ResNet	110	1.7M	6.43 (6.61±0.16)
ResNet	1202	19.4M	7.93

Table 6. Classification error on the **CIFAR-10** test set. All methods are with data augmentation. For ResNet-110, we run it 5 times and show “best (mean±std)” as in [43].

so our residual models have exactly the same depth, width, and number of parameters as the plain counterparts.

We use a weight decay of 0.0001 and momentum of 0.9, and adopt the weight initialization in [13] and BN [16] but with no dropout. These models are trained with a mini-batch size of 128 on two GPUs. We start with a learning rate of 0.1, divide it by 10 at 32k and 48k iterations, and terminate training at 64k iterations, which is determined on a 45k/5k train/val split. We follow the simple data augmentation in [24] for training: 4 pixels are padded on each side, and a 32×32 crop is randomly sampled from the padded image or its horizontal flip. For testing, we only evaluate the single view of the original 32×32 image.

We compare $n = \{3, 5, 7, 9\}$, leading to 20, 32, 44, and 56-layer networks. Fig. 6 (left) shows the behaviors of the plain nets. The deep plain nets suffer from increased depth, and exhibit higher training error when going deeper. This phenomenon is similar to that on ImageNet (Fig. 4, left) and on MNIST (see [42]), suggesting that such an optimization difficulty is a fundamental problem.

Fig. 6 (middle) shows the behaviors of ResNets. Also similar to the ImageNet cases (Fig. 4, right), our ResNets manage to overcome the optimization difficulty and demonstrate accuracy gains when the depth increases.

We further explore $n = 18$ that leads to a 110-layer ResNet. In this case, we find that the initial learning rate of 0.1 is slightly too large to start converging⁵. So we use 0.01 to warm up the training until the training error is below 80% (about 400 iterations), and then go back to 0.1 and continue training. The rest of the learning schedule is as done previously. This 110-layer network converges well (Fig. 6, middle). It has *fewer* parameters than other deep and thin

⁵With an initial learning rate of 0.1, it starts converging (<90% error) after several epochs, but still reaches similar accuracy.

通过采用这种3层瓶颈块替换34层网络中的2层块，最终得到了50层ResNet网络（表1）。为提升网络的复杂度，我们采用了选项B。该模型需要执行38亿次浮点运算次数才能完成计算。**101层结构与152层结构的ResNet网络：**我们通过使用更多的3层结构块来构建101层结构和152层结构的ResNet网络（见表1）。值得注意的是，尽管这些网络的深度大幅增加，但152层结构的ResNet网络（浮点运算次数为113亿次）的复杂度依然低于VGG-16/19网络（浮点运算次数分别为153亿次和196亿次）。

50层、101层和152层结构的ResNet网络的准确率比34层结构的同类网络高出不少幅度（见表3和表4）。我们并未观察到性能退化问题，因此随着网络深度的显著增加，其准确率也获得了大幅提升。在所有评估指标上都能体现出深度带来的优势（见表3和表4）。

与最先进方法的对比。在表4中，我们将自己的模型与此前最佳的单一模型结果进行了比较。作为基准的34层结构ResNet网络已经取得了极具竞争力的准确率。而我们的152层结构ResNet网络在单一模型测试中的前5错误率为4.49%。这一单一模型结果甚至优于此前所有的集成模型结果（见表5）。我们还将六个不同深度的模型组合在一起形成了一个集成模型（提交时仅使用了两个152层结构的模型），该集成模型在测试集上的前5错误率为**3.57%**（见表5）。该成果在*ILSVRC2015*竞赛中获得了第一名。

4.2. CIFAR-10数据集与分析

我们对CIFAR-10数据集进行了更多研究[20], , 该数据集包含5万张训练图像和1万张测试图像，共分为10个类别。我们展示了在训练集上训练、在测试集上评估的实验结果。我们的重点在于研究极深神经网络的行为特征，而非追求最先进结果，因此特意采用了以下较为简单的架构。

这些普通型/残差型架构的结构如图3（中间/右侧）所示。神经网络的输入为 32×32 图像，且已减去每像素的均值。第一层是 3×3 卷积层。随后，我们在不同尺寸的 $\{32, 16, 8\}$ 特征图上分别使用由 $6n$ 层组成的堆叠结构，每种特征图尺寸对应 $2n$ 层，各层的滤波器数量分别为 $\{16, 32, 64\}$ 。降采样是通过步长为2的卷积来实现的。整个网络最后通过全局平均池化、一个10路的全连接层以及softmax函数完成输出。总共共有 $6n+2$ 层经过加权处理的堆叠层。下表总结了该架构的详细信息：

输出映射尺寸	32×32	16×16	8×8
# 层数	$1+2n$	$2n$	$2n$
# 过滤器数量	16	32	64

当使用快捷连接时，这些连接会被接至 3×3 层之间的对应组合上（共计 $3n$ 条快捷连接）。在本数据集的所有案例中，我们均采用恒等快捷路径（即选项A），

方法	误差 (%)		
最大值输出算法 [10]	9.38		
NIN [25]	8.81		
DSN [24]	8.22		
	# 层	# 参数数量	
FitNet [35]	19	2.5M	8.39
Highway模型 [42, 43]	19	2.3M	7.54 (7.72±0.16)
Highway模型 [42, 43]	32	125万	8.80
ResNet网络	20	27万	8.75
ResNet网络	32	46万	7.51
ResNet网络	44	66万	7.17
ResNet网络	56	85万	6.97
ResNet网络	110	1.7M	6.43 (6.61±0.16)
ResNet	1202	1940万	7.93

表6: **CIFAR-10** 测试集上的分类误差。所有方法均采用了数据增强技术。对于ResNet-110模型，我们进行了5次训练，并如 [43]所示，仅展示“最佳值（均值±标准差）”。

因此，我们的残差模型的深度、宽度以及参数数量都与对应的普通模型完全相同。

我们在训练中设置了0.0001的权重衰减系数和0.9的动量值，同时采用 [13] 中的权重初始化方式以及 [16] 中的批归一化处理，但不使用丢弃机制。这些模型在两块图形处理单元上以128的迷你批量大小进行训练。初始学习率为0.1，在训练到32千次和48千次迭代时分别将学习率降低为原来的1/10，而在基于45千次训练数据、5千次验证数据设定的64千次迭代时终止训练。在训练过程中，我们采用 [24] 中描述的简单数据增强手段：在图像的四周各填充4像素，然后从经过填充的图像或其水平翻转版本中随机选取一个 32×32 大小的区域进行裁剪。至于测试阶段，我们仅对原始 32×32 图像的单一视图进行评估。

我们对比了 $n = \{3, 5, 7, 9\}$ ，由此得到了具有20层、32层、44层以及56层结构的神经网络。图6左侧展示了普通网络的表现情况——随着层次深度的增加，这类深层普通网络会面临更大的优化难题，其训练误差也会随之上升。这一现象与在 ImageNet数据集（图4左侧）以及MNIST数据集中的情况类似（参见 [42]），这说明此类优化难题其实是一个根本性问题。

图6中间部分展示了ResNet系列模型的表现。与ImageNet数据集中的情况类似（图4右侧），我们的ResNet网络也能够克服这种优化难题，即便层次深度增加，依然能够实现准确率提升。

我们进一步研究了能够构建110层ResNet网络的 $n = 18$ 方法。实验表明，初始学习率设为0.1时略高，难以让模型开始收敛⁵。因此我们先使用0.01的较低学习率进行训练预热，直到训练误差降至80%以下（大约经过400次迭代），之后再将学习率恢复至0.1并继续训练。其余的学习计划与之前的方案一致。该110层神经网络收敛效果良好（见图6中间部分）。与其他深度或浅层的神经网络相比，它的参数数量更少，但仍能取得当前最佳性能，其对应的最先进结果为6.43%（见表6）。

⁵在初始学习率为0.1的情况下，这类神经网络经过几轮训练后便开始收敛，此时的训练误差为<90%，不过最终达到的准确率仍相当接近。

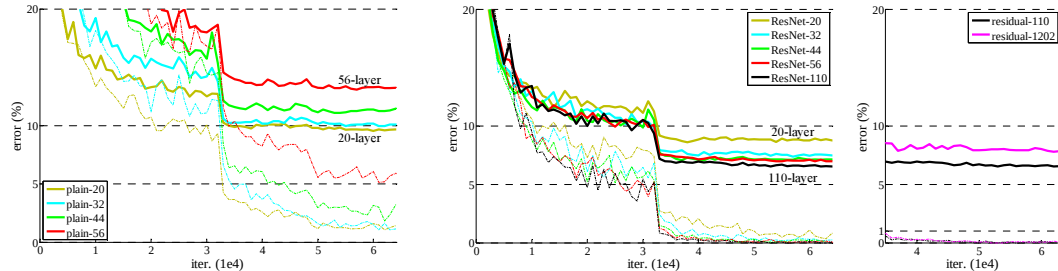


Figure 6. Training on **CIFAR-10**. Dashed lines denote training error, and bold lines denote testing error. **Left**: plain networks. The error of plain-110 is higher than 60% and not displayed. **Middle**: ResNets. **Right**: ResNets with 110 and 1202 layers.

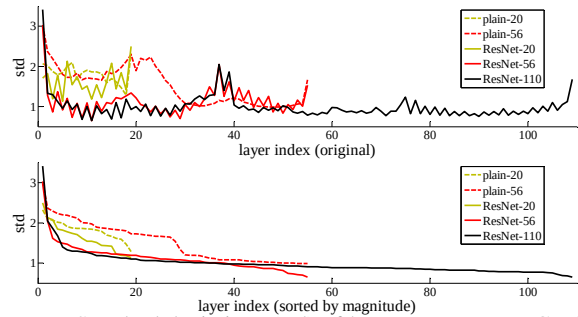


Figure 7. Standard deviations (std) of layer responses on CIFAR-10. The responses are the outputs of each 3×3 layer, after BN and before nonlinearity. **Top**: the layers are shown in their original order. **Bottom**: the responses are ranked in descending order.

networks such as FitNet [35] and Highway [42] (Table 6), yet is among the state-of-the-art results (6.43%, Table 6).

Analysis of Layer Responses. Fig. 7 shows the standard deviations (std) of the layer responses. The responses are the outputs of each 3×3 layer, after BN and before other nonlinearity (ReLU/addition). For ResNets, this analysis reveals the response strength of the residual functions. Fig. 7 shows that ResNets have generally smaller responses than their plain counterparts. These results support our basic motivation (Sec.3.1) that the residual functions might be generally closer to zero than the non-residual functions. We also notice that the deeper ResNet has smaller magnitudes of responses, as evidenced by the comparisons among ResNet-20, 56, and 110 in Fig. 7. When there are more layers, an individual layer of ResNets tends to modify the signal less.

Exploring Over 1000 layers. We explore an aggressively deep model of over 1000 layers. We set $n = 200$ that leads to a 1202-layer network, which is trained as described above. Our method shows *no optimization difficulty*, and this 10^3 -layer network is able to achieve *training error* $< 0.1\%$ (Fig. 6, right). Its test error is still fairly good (7.93%, Table 6).

But there are still open problems on such aggressively deep models. The testing result of this 1202-layer network is worse than that of our 110-layer network, although both

training data	07+12	07++12
test data	VOC 07 test	VOC 12 test
VGG-16	73.2	70.4
ResNet-101	76.4	73.8

Table 7. Object detection mAP (%) on the PASCAL VOC 2007/2012 test sets using **baseline** Faster R-CNN. See also Table 10 and 11 for better results.

metric	mAP@.5	mAP@[.5, .95]
VGG-16	41.5	21.2
ResNet-101	48.4	27.2

Table 8. Object detection mAP (%) on the COCO validation set using **baseline** Faster R-CNN. See also Table 9 for better results.

have similar training error. We argue that this is because of overfitting. The 1202-layer network may be unnecessarily large (19.4M) for this small dataset. Strong regularization such as maxout [10] or dropout [14] is applied to obtain the best results ([10, 25, 24, 35]) on this dataset. In this paper, we use no maxout/dropout and just simply impose regularization via deep and thin architectures by design, without distracting from the focus on the difficulties of optimization. But combining with stronger regularization may improve results, which we will study in the future.

4.3. Object Detection on PASCAL and MS COCO

Our method has good generalization performance on other recognition tasks. Table 7 and 8 show the object detection baseline results on PASCAL VOC 2007 and 2012 [5] and COCO [26]. We adopt *Faster R-CNN* [32] as the detection method. Here we are interested in the improvements of replacing VGG-16 [41] with ResNet-101. The detection implementation (see appendix) of using both models is the same, so the gains can only be attributed to better networks. Most remarkably, on the challenging COCO dataset we obtain a 6.0% increase in COCO’s standard metric (mAP@[.5, .95]), which is a 28% relative improvement. This gain is solely due to the learned representations.

Based on deep residual nets, we won the 1st places in several tracks in ILSVRC & COCO 2015 competitions: ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation. The details are in the appendix.

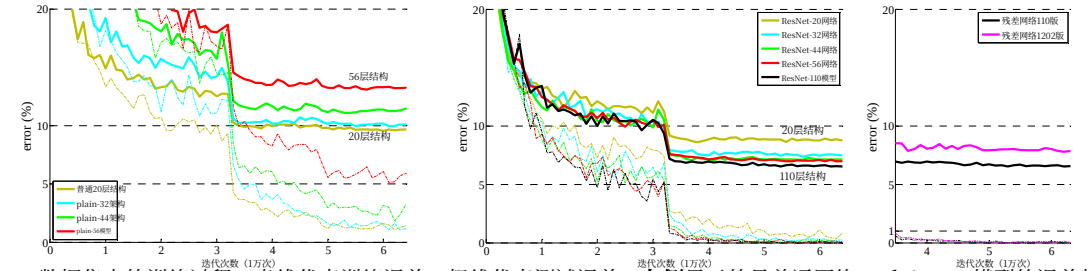


图6: **CIFAR-10**数据集上的训练过程。虚线代表训练误差, 粗线代表测试误差。左侧显示的是普通网络, plain-110模型的误差超过60%, 因此未在图中展示。中间为ResNets系列模型, 右侧则为具有110层和1202层的ResNets模型。

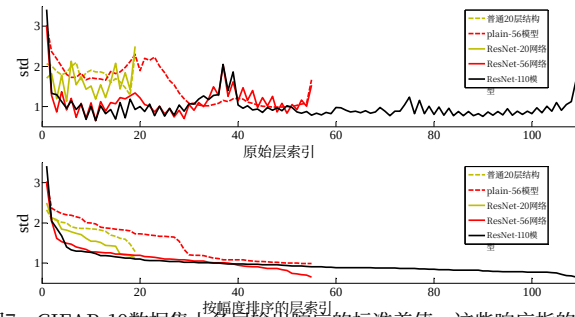


图7: CIFAR-10数据集上各层输出响应的标准差值。这些响应指的是经过批归一化处理后、但在应用非线性运算之前的每一 3×3 层的输出值。顶部的图表按层的原始顺序展示, 而底部的图表则按照响应强度由大到小的顺序排列。

尽管它不如FitNet [35]和Highway [42] (见表6) 这类神经网络, 但其性能依然属于当前最佳性能范畴, 对应的准确率为6.43%。

层输出响应的分析。如图7所示, 这些数值为各层输出响应的标准差值。所谓响应值, 是指经过批归一化处理之后、在随后应用其他非线性运算(如ReLU函数或加法运算)之前的每一 3×3 层的输出结果。对于ResNets网络, 这项分析能够揭示出其中残差函数的响应强度。图7显示, ResNets网络的响应值通常比对应的普通网络更小。这些结果印证了我们在3.1节中提出的基本假设, 即残差函数的数值往往比非残差函数的数值更接近于零。此外, 从图7中ResNet-20、56和110模型之间的对比可以看出, 层数越多的ResNet网络, 其各层输出的响应幅度就越小; 层数增多意味着单个层对信号的处理影响相对减弱。

探索超过1000层的结构。我们研究了一种层数极为庞大的模型, 其层数超过了1000层。我们设定 $n = 200$, 最终构建出了一个包含1202层的神经网络, 并按照前述方法对其进行训练。实验结果表明, 不存在优化难题, 该 10^3 层结构的神经网络能够将训练误差控制在 $< 0.1\%$ 的水平(见图6右侧)。其在测试数据上的误差也相当理想, 仅为7.93%(见表6)。

不过, 对于如此深度极高的模型而言, 仍然存在一些尚未解决的问题。尽管这两者具有相似的训练误差, 但这个1202层网络的测试结果却比我们的110层网络更差。

训练数据	07+12	07++12
测试数据	VOC 07测试集	PASCAL VOC 2007/2012测试集
VGG-16网络	73.2	70.4
ResNet-101网络	76.4	73.8

表7: 使用**基准模型**Faster R-CNN在PASCAL VOC 2007/2012测试集上进行的物体检测mAP (%)数值。如需更详细的分析结果, 请参考表10和表11。

评估指标	mAP@.5指标	mAP@[.5, .95]
VGG-16网络	41.5	21.2
ResNet-101网络	48.4	27.2

表8显示了使用**基准模型**Faster R-CNN在COCO验证集上进行的物体检测的mAP值 (%)。如需更优的检测结果, 请参考表9。

我们认为这是因为出现了过拟合现象。对于这样规模较小的数据集而言, 1202层网络的结构可能过于庞大(参数量高达1940万)。为此, 我们采用了最大值输出算法[10]或丢弃机制[14]等强正则化手段, 以期在该数据集上获得最佳效果([10, 25, 24, 35])。在本文中, 我们并未使用最大值输出算法或丢弃机制, 而是通过精心设计的深浅层次结构来实现正则化, 从而避免分散对优化难题的关注。当然, 结合更强的正则化方法或许能进一步提升性能, 这是我们未来将要研究的方向。

4.3. PASCAL与MS COCO数据集上的物体检测

我们的方法在其他识别任务中也表现出良好的泛化性能。表7和表8分别展示了在PASCAL VOC 2007、2012[5]以及COCO [26]数据集上采用物体检测基准模型所获得的检测结果。我们选用*Faster R-CNN* [32]作为检测方法。在此研究中, 我们重点关注用ResNet-101替代VGG-16 [41]所带来的改进效果。由于两种模型对应的检测实现方式(详见附录)完全相同, 因此这些提升只能归因于更优秀的神经网络结构。最令人瞩目的是, 在难度较高的COCO数据集上, 我们的方法使得该数据集的标准评估指标mAP@[.5, .95]提升了6.0%, 相当于相对提升了28%。这一显著提升完全得益于优化后的特征表示能力。

依托深度残差网络, 我们在ILSVRC与COCO 2015竞赛的多个赛道中取得了第一名, 这些赛道包括ImageNet检测任务、ImageNet定位任务、COCO检测任务以及COCO分割任务。详细内容请参见附录。

References

- [1] Y. Bengio, P. Simard, and P. Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994.
- [2] C. M. Bishop. *Neural networks for pattern recognition*. Oxford university press, 1995.
- [3] W. L. Briggs, S. F. McCormick, et al. *A Multigrid Tutorial*. Siam, 2000.
- [4] K. Chatfield, V. Lempitsky, A. Vedaldi, and A. Zisserman. The devil is in the details: an evaluation of recent feature encoding methods. In *BMVC*, 2011.
- [5] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman. The Pascal Visual Object Classes (VOC) Challenge. *IJCV*, pages 303–338, 2010.
- [6] S. Gidaris and N. Komodakis. Object detection via a multi-region & semantic segmentation-aware cnn model. In *ICCV*, 2015.
- [7] R. Girshick. Fast R-CNN. In *ICCV*, 2015.
- [8] R. Girshick, J. Donahue, T. Darrell, and J. Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *CVPR*, 2014.
- [9] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *AISTATS*, 2010.
- [10] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio. Maxout networks. *arXiv:1302.4389*, 2013.
- [11] K. He and J. Sun. Convolutional neural networks at constrained time cost. In *CVPR*, 2015.
- [12] K. He, X. Zhang, S. Ren, and J. Sun. Spatial pyramid pooling in deep convolutional networks for visual recognition. In *ECCV*, 2014.
- [13] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *ICCV*, 2015.
- [14] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv:1207.0580*, 2012.
- [15] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [16] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.
- [17] H. Jegou, M. Douze, and C. Schmid. Product quantization for nearest neighbor search. *TPAMI*, 33, 2011.
- [18] H. Jegou, F. Perronnin, M. Douze, J. Sanchez, P. Perez, and C. Schmid. Aggregating local image descriptors into compact codes. *TPAMI*, 2012.
- [19] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell. Caffe: Convolutional architecture for fast feature embedding. *arXiv:1408.5093*, 2014.
- [20] A. Krizhevsky. Learning multiple layers of features from tiny images. *Tech Report*, 2009.
- [21] A. Krizhevsky, I. Sutskever, and G. Hinton. Imagenet classification with deep convolutional neural networks. In *NIPS*, 2012.
- [22] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1989.
- [23] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller. Efficient backprop. In *Neural Networks: Tricks of the Trade*, pages 9–50. Springer, 1998.
- [24] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang, and Z. Tu. Deeply-supervised nets. *arXiv:1409.5185*, 2014.
- [25] M. Lin, Q. Chen, and S. Yan. Network in network. *arXiv:1312.4400*, 2013.
- [26] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *ECCV*. 2014.
- [27] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. In *CVPR*, 2015.

- [28] G. Montúfar, R. Pascanu, K. Cho, and Y. Bengio. On the number of linear regions of deep neural networks. In *NIPS*, 2014.
- [29] V. Nair and G. E. Hinton. Rectified linear units improve restricted boltzmann machines. In *ICML*, 2010.
- [30] F. Perronnin and C. Dance. Fisher kernels on visual vocabularies for image categorization. In *CVPR*, 2007.
- [31] T. Raiko, H. Valpola, and Y. LeCun. Deep learning made easier by linear transformations in perceptrons. In *AISTATS*, 2012.
- [32] S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *NIPS*, 2015.
- [33] S. Ren, K. He, R. Girshick, X. Zhang, and J. Sun. Object detection networks on convolutional feature maps. *arXiv:1504.06066*, 2015.
- [34] B. D. Ripley. *Pattern recognition and neural networks*. Cambridge university press, 1996.
- [35] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio. Fitnets: Hints for thin deep nets. In *ICLR*, 2015.
- [36] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al. Imagenet large scale visual recognition challenge. *arXiv:1409.0575*, 2014.
- [37] A. M. Saxe, J. L. McClelland, and S. Ganguli. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. *arXiv:1312.6120*, 2013.
- [38] N. N. Schraudolph. Accelerated gradient descent by factor-centering decomposition. Technical report, 1998.
- [39] N. N. Schraudolph. Centering neural network gradient factors. In *Neural Networks: Tricks of the Trade*, pages 207–226. Springer, 1998.
- [40] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun. Overfeat: Integrated recognition, localization and detection using convolutional networks. In *ICLR*, 2014.
- [41] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.
- [42] R. K. Srivastava, K. Greff, and J. Schmidhuber. Highway networks. *arXiv:1505.00387*, 2015.
- [43] R. K. Srivastava, K. Greff, and J. Schmidhuber. Training very deep networks. *1507.06228*, 2015.
- [44] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. In *CVPR*, 2015.
- [45] R. Szeliski. Fast surface interpolation using hierarchical basis functions. *TPAMI*, 1990.
- [46] R. Szeliski. Locally adapted hierarchical basis preconditioning. In *SIGGRAPH*, 2006.
- [47] T. Vatanen, T. Raiko, H. Valpola, and Y. LeCun. Pushing stochastic gradient towards second-order methods—backpropagation learning with transformations in nonlinearities. In *Neural Information Processing*, 2013.
- [48] A. Vedaldi and B. Fulkerson. VLFeat: An open and portable library of computer vision algorithms, 2008.
- [49] W. Venables and B. Ripley. Modern applied statistics with s-plus. 1999.
- [50] M. D. Zeiler and R. Fergus. Visualizing and understanding convolutional neural networks. In *ECCV*, 2014.

参考文献

- [1] Y. Bengio, P. Simard, 和 P. Frasconi. 利用梯度下降学习长期依赖关系十分困难。《IEEE神经网络汇刊》, 5(2):157–166, 1994年。[2] C. M. Bishop. 《用于模式识别的神经网络》。牛津大学出版社, 1995年。[3] W. L. Briggs, S. F. McCormick等人. 《多重网格教程》。Siam出版社, 2000年。[4] K. Chatfield, V. Lempitsky, A. Vedaldi和A. Zisserman. “细节决定成败：近期特征编码方法的评估”。载于BMVC会议, 2011年。[5] M. Everingham, L. Van Gool, C. K. Williams, J. Winn和A. Zisserman. “Pascal视觉对象类别 (VOC) 挑战赛”。IJCV会议, 第 303–338页, 2010年。[6] S. Gidaris和N. Komodakis. “基于多区域及语义分割感知的CNN模型进行物体检测”。载于ICCV会议, 2015年。[7] R. Girshick. “Fast R-CNN算法”。载于ICCV会议, 2015年。[8] R. Girshick, J. Donahue, T. Darrell和J. Malik. “用于精确物体检测与语义分割的丰富特征层次结构”。载于CVPR会议, 2014年。[9] X. Glorot和Y. Bengio. “理解训练深度前馈神经网络的难度”。载于AISTATS会议, 2010年。[10] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville和Y. Bengio. “Maxout网络”。arXiv:1302.4389, 2013年。[11] K. He和J. Sun. “在时间成本受限条件下的卷积神经网络”。载于CVPR会议, 2015年。[12] K. He, X. Zhang, S. Ren和J. Sun. “深度卷积网络中的空间金字塔池化技术及其在视觉识别中的应用”。载于ECCV会议, 2014年。[13] K. He, X. Zhang, S. Ren和J. Sun. “深入研究整流器：在ImageNet分类任务上超越人类水平的表现”。载于ICCV会议, 2015年。[14] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever和R. R. Salakhutdinov. “通过防止特征检测器的共同适应来提升神经网络性能”。arXiv:1207.0580, 2012年。[15] S. Hochreiter和J. Schmidhuber. “长短期记忆”。《神经计算》期刊, 9(8):1735–1780, 1997年。[16] S. Ioffe和C. Szegedy. “批量归一化：通过减少内部协变量偏移来加速深度网络训练”。载于ICML会议, 2015年。[17] H. Jegou, M. Douze和C. Schmid. “用于最近邻搜索的乘积量化技术”。《TPAMI》期刊, 33卷, 2011年。[18] H. Jegou, F. Perronnin, M. Douze, J. Sanchez, P. Perez和C. Schmid. “将局部图像描述符聚合为紧凑编码”。《TPAMI》期刊, 2012年。[19] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama和T. Darrell. “Caffe：用于快速特征嵌入的卷积架构”。arXiv:1408.5093, 2014年。[20] A. Krizhevsky. “从极小尺寸的图像中学习多层特征”。技术报告, 2009年。[21] A. Krizhevsky, I. Sutskever和G. Hinton. “利用深度卷积神经网络进行ImageNet分类”。载于NIPS会议, 2012年。[22] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard和L. D. Jackel. “反向传播算法在手写邮政编码识别中的应用”。《神经计算》期刊, 1989年。[23] Y. LeCun, L. Bottou, G. B. Orr和K.-R. Müller. “高效反向传播算法”。载于《神经网络：行业技巧》一书, 第9–50页。Springer出版社, 1998年。[24] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang和Z. Tu. “深度监督网络”。arXiv:1409.5185, 2014年。[25] M. Lin, Q. Chen和S. Yan. “网络中的网络”。arXiv:1312.4400, 2013年。[26] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár和C. L. Zitnick. “Microsoft COCO：上下文中的常见物体”。载于ECCV会议, 2014年。[27] J. Long, E. Shelhamer和T. Darrell. “用于语义分割的全卷积网络”。载于CVPR会议, 2015年。

- [28] G. Mont’ufar, R. Pascanu, K. Cho以及Y. Bengio共同发表了《关于深度神经网络中线性区域的数量》一文, 该文发表于2014年的NIPS会议上。[29] V. Nair与G. E. Hinton在2010年的ICML会议上提出了“修正线性单元可提升受限玻尔兹曼机的性能”这一观点。[30] F. Perronnin和C. Dance则在2007年的CVPR会议上介绍了基于视觉词汇表的费希尔核方法, 用于图像分类任务。[31] T. Raiko, H. Valpola和Y. LeCun于2012年在AISTATS会议上撰文, 指出通过在感知器中应用线性变换能够简化深度学习流程。[32] S. Ren, K. He, R. Girshick和J. Sun在2015年的NIPS会议上发布了Faster R-CNN, 该算法借助区域提议网络实现了实时物体检测功能。[33] 随后在2015年, 同一团队又在同一会议上发表了另一篇论文, 主题为“基于卷积特征图的物体检测网络”, 相关研究成果收录在arXiv预印本平台上的论文编号为1504.06066。[34] B. D. Ripley所著的《模式识别与神经网络》一书由剑桥大学出版社于1996年出版。[35] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta以及Y. Bengio在2015年的ICLR会议上提出了Fitnets算法, 旨在为结构较薄的深度神经网络提供优化思路。[36] O. Russakovsky等人于2014年在arXiv上发布了Imagenet大规模视觉识别挑战赛的相关资料, 论文编号为1409.0575。[37] A. M. Saxe, J. L. McClelland和S. Ganguli在2013年的arXiv上发表了论文, 给出了深度线性神经网络中学习过程非线性动力学的精确解, 对应论文编号为1312.6120。[38] N. N. Schraudolph在1998年撰写了一份技术报告, 提出了通过因子中心分解加速梯度下降的方法。[39] 同年, 他在Springer出版的《神经网络：行业技巧》一书中, 于第207至226页详细阐述了如何对神经网络梯度因子进行归一化处理, 该书论文编号为1998。[40] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus和Y. Le-Cun在2014年的ICLR会议上介绍了Overfeat算法, 这种算法能够利用卷积神经网络同时实现物体的识别、定位与检测功能。[41] K. Simonyan和A. Zisserman在2015年的ICLR会议上发表了关于使用超深层卷积神经网络进行大规模图像识别的研究。[42] R. K. Srivastava, K. Greff和J. Schmidhuber在2015年的arXiv上发布了Highway网络的相关研究, 论文编号为1505.00387; 同年他们还发表了另一篇关于训练超深层神经网络的研究, 论文编号为1507.06228。[43] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke和A. Rabinovich在2015年的CVPR会议上发表了《通过卷积进一步加深网络深度》一文。[44] R. Szeliski在1990年的TPAMI期刊上发表了《利用分层基函数实现快速表面插值》一文。[45] 他在2006年的SIGGRAPH会议上又发表了题为《局部自适应的分层基预处理》的研究成果。[46] T. Vatanen, T. Raiko, H. Valpola和Y. LeCun在2013年的《神经信息处理领域相关研究》会议上提出了将随机梯度方法向二阶方法推进的理念, 即通过非线性变换结合反向传播算法来实现学习。[47] A. Vedaldi和B. Fulkerson在2008年开发了VLFeat, 这是一个开放且易于移植的计算机视觉算法库。[48] W. Venables和B. Ripley在1999年推出了结合s-plus工具的现代应用统计学教材。[49] M. D. Zeiler和R. Fergus在2014年的ECCV会议上发表了《可视化与理解卷积神经网络》一文。

A. Object Detection Baselines

In this section we introduce our detection method based on the baseline Faster R-CNN [32] system. The models are initialized by the ImageNet classification models, and then fine-tuned on the object detection data. We have experimented with ResNet-50/101 at the time of the ILSVRC & COCO 2015 detection competitions.

Unlike VGG-16 used in [32], our ResNet has no hidden fc layers. We adopt the idea of “Networks on Conv feature maps” (NoC) [33] to address this issue. We compute the full-image shared conv feature maps using those layers whose strides on the image are no greater than 16 pixels (*i.e.*, conv1, conv2_x, conv3_x, and conv4_x, totally 91 conv layers in ResNet-101; Table 1). We consider these layers as analogous to the 13 conv layers in VGG-16, and by doing so, both ResNet and VGG-16 have conv feature maps of the same total stride (16 pixels). These layers are shared by a region proposal network (RPN, generating 300 proposals) [32] and a Fast R-CNN detection network [7]. RoI pooling [7] is performed before conv5_1. On this RoI-pooled feature, all layers of conv5_x and up are adopted for each region, playing the roles of VGG-16’s fc layers. The final classification layer is replaced by two sibling layers (classification and box regression [7]).

For the usage of BN layers, after pre-training, we compute the BN statistics (means and variances) for each layer on the ImageNet training set. Then the BN layers are fixed during fine-tuning for object detection. As such, the BN layers become linear activations with constant offsets and scales, and BN statistics are not updated by fine-tuning. We fix the BN layers mainly for reducing memory consumption in Faster R-CNN training.

PASCAL VOC

Following [7, 32], for the PASCAL VOC 2007 *test* set, we use the 5k *trainval* images in VOC 2007 and 16k *trainval* images in VOC 2012 for training (“07+12”). For the PASCAL VOC 2012 *test* set, we use the 10k *trainval+test* images in VOC 2007 and 16k *trainval* images in VOC 2012 for training (“07++12”). The hyper-parameters for training Faster R-CNN are the same as in [32]. Table 7 shows the results. ResNet-101 improves the mAP by >3% over VGG-16. This gain is solely because of the improved features learned by ResNet.

MS COCO

The MS COCO dataset [26] involves 80 object categories. We evaluate the PASCAL VOC metric (mAP @ IoU = 0.5) and the standard COCO metric (mAP @ IoU = .5:.05:.95). We use the 80k images on the train set for training and the 40k images on the val set for evaluation. Our detection system for COCO is similar to that for PASCAL VOC. We train the COCO models with an 8-GPU implementation, and thus the RPN step has a mini-batch size of

8 images (*i.e.*, 1 per GPU) and the Fast R-CNN step has a mini-batch size of 16 images. The RPN step and Fast R-CNN step are both trained for 240k iterations with a learning rate of 0.001 and then for 80k iterations with 0.0001.

Table 8 shows the results on the MS COCO validation set. ResNet-101 has a 6% increase of mAP@[.5, .95] over VGG-16, which is a 28% relative improvement, solely contributed by the features learned by the better network. Remarkably, the mAP@[.5, .95]’s absolute increase (6.0%) is nearly as big as mAP@.5’s (6.9%). This suggests that a deeper network can improve both recognition and localization.

B. Object Detection Improvements

For completeness, we report the improvements made for the competitions. These improvements are based on deep features and thus should benefit from residual learning.

MS COCO

Box refinement. Our box refinement partially follows the iterative localization in [6]. In Faster R-CNN, the final output is a regressed box that is different from its proposal box. So for inference, we pool a new feature from the regressed box and obtain a new classification score and a new regressed box. We combine these 300 new predictions with the original 300 predictions. Non-maximum suppression (NMS) is applied on the union set of predicted boxes using an IoU threshold of 0.3 [8], followed by box voting [6]. Box refinement improves mAP by about 2 points (Table 9).

Global context. We combine global context in the Fast R-CNN step. Given the full-image conv feature map, we pool a feature by global Spatial Pyramid Pooling [12] (with a “single-level” pyramid) which can be implemented as “RoI” pooling using the entire image’s bounding box as the RoI. This pooled feature is fed into the post-RoI layers to obtain a global context feature. This global feature is concatenated with the original per-region feature, followed by the sibling classification and box regression layers. This new structure is trained end-to-end. Global context improves mAP@.5 by about 1 point (Table 9).

Multi-scale testing. In the above, all results are obtained by single-scale training/testing as in [32], where the image’s shorter side is $s = 600$ pixels. Multi-scale training/testing has been developed in [12, 7] by selecting a scale from a feature pyramid, and in [33] by using maxout layers. In our current implementation, we have performed multi-scale *testing* following [33]; we have not performed multi-scale training because of limited time. In addition, we have performed multi-scale testing only for the Fast R-CNN step (but not yet for the RPN step). With a trained model, we compute conv feature maps on an image pyramid, where the image’s shorter sides are $s \in \{200, 400, 600, 800, 1000\}$.

A. 对象检测基准模型

在本节中，我们将介绍基于基准模型FasterR-CNN [32]系统的物体检测方法。这些模型首先会利用ImageNet分类模型进行初始化，之后再在物体检测数据集上开展微调训练。在参与ILSVRC及COCO 2015物体检测竞赛期间，我们尝试使用了ResNet-50/101这类深度神经网络作为检测模型的核心结构。

与 [32], 中使用的VGG-16不同，我们的ResNet没有隐藏的全连接层。为了解决这一问题，我们采用了“基于卷积特征图的神经网络”（NoC机制） [33]。我们利用那些在图像上的步长不超过16像素的层来计算全图像共享的卷积特征图（即，包括conv1层、conv2x层、conv3x层以及conv4x层，ResNet-101中共有91个这样的卷积层；详见表1）。我们将这些层视为与VGG-16中的13个卷积层相当，这样一来，ResNet和VGG-16的卷积特征图的总步长都为16像素。这些特征图会被区域提议网络（RPN，可生成300个候选区域） [32] 以及Fast R-CNN检测网络 [7]共同使用。在conv5_1层之前会先进行ROI池化 [7] 操作。在经过ROI池化处理后的特征图上，每个区域都会使用从conv5x层开始的后续所有层，这些层承担着与VGG-16全连接层相同的功能。最终的分层则被两个功能相近的层——分类层和边界框回归层 [7]——所替代。

在使用批归一化层时，预训练完成后，我们会基于ImageNet训练集为每一层计算批归一化的统计量（均值和方差）。在后续用于目标检测的微调过程中，这些批归一化层会被固定不变。这样一来，它们就变成了具有恒定偏移量和缩放因子的线性激活函数，且其批归一化统计量也不会微调过程中被更新。我们设置批归一化层为固定状态，主要是为了降低Faster R-CNN训练时的内存消耗。

PASCAL VOC

针对PASCAL VOC 2007的测试集，我们采用了VOC 2007中的5千张训练/验证图像以及VOC 2012中的1.6万张训练/验证图像来进行训练（“07+12”）。而对于PASCAL VOC 2012的测试集，则使用VOC 2007中的1万张训练/验证+测试图像以及VOC 2012中的1.6万张训练/验证图像进行训练（“07++12”）。用于训练Faster R-CNN的超参数与 [32] 中的一致。表7展示了相应的实验结果：相比VGG-16，ResNet-101能使mAP值提升 >3%，这一提升完全得益于ResNet所学习到的更优特征。

MS COCO数据集

MS COCO数据集 [26] 涵盖了80种不同的物体类别。我们采用PASCAL VOC的评估指标（mAP @ IoU = 0.5）以及标准的COCO评估指标（mAP @ IoU =.5:.05:.95）来进行性能测试。训练过程中我们会使用训练集中的8万张图像，而评估则使用验证集中的4万张图像。针对COCO设计的检测系统与用于PASCAL VOC的系统类似。我们在配备8个图形处理单元的架构上训练COCO模型，因此区域提议网络阶段的每小批量大小为8张图像（即，每个图形处理单元对应1张），而Fast R-CNN阶段的每小批量大小则为16张图像。区域提议网络阶段和Fast R-CNN阶段均以0.001的学习率进行24万次迭代训练，之后再以0.0001的学习率进行8万次迭代训练。

区域提议网络阶段和Fast R-CNN阶段的训练次数均为240万次，学习率初始设定为0.001；完成该阶段训练后，两者会继续以0.0001的学习率进行8万次迭代训练。

表8展示了在MS COCO验证集上的测试结果。与VGG-16相比，ResNet-101的mAP@[.5, .95]提升了6%，相对提升幅度达28%，这一提升完全得益于该更优神经网络所学习到的特征。值得注意的是，mAP@[.5, .95]的绝对提升幅度（6.0%）几乎与mAP@.5的提升幅度（6.9%）相当。这表明更深层的神经网络能够同时提升物体的识别精度与定位能力。

B. 物体检测性能的提升

为保证信息的完整性，我们在此汇报在各类比赛中取得的性能提升成果。这些提升得益于深度特征的应用，因此自然也能从残差学习中受益。

MS COCO数据集

框精炼。我们的框精炼方法部分借鉴了 [6]中提到的迭代定位技术。在Faster R-CNN中，最终输出的是经过回归处理后的框，这与初始提出的框并不相同。因此在推理阶段，我们会从回归后的框中提取新的特征，进而得到新的分类分数和新的回归框。随后我们将这300个新的预测结果与最初的300个预测结果相结合，对所有预测框的并集应用非最大值抑制算法，其IoU阈值设为0.3 [8]，之后再采用框投票算法 [6]。实验表明，框精炼可使mAP值提升约2分（见表9）。

上下文信息。我们在Fast R-CNN步骤中整合了上下文信息。基于整幅图像的卷积特征图，我们通过全局空间金字塔池化 [12]（采用“单层”金字塔结构）对特征进行池化处理，该操作实际上可以视为以整幅图像的边界框作为感兴趣区域来执行“RoI”池化。经过池化后的特征会被输入到RoI之后的层中，从而生成上下文特征。这个全局特征会与最初的各区域特征相串联，随后再传递给分类层和边界框回归层。整个新架构会通过端到端的方式共同训练，而上下文信息的引入能够让mAP@.5提升约1分（见表9）。</solution>

进行多尺度检测。如上所述，所有结果均是通过单尺度训练/测试获得的，其参数设置与 [32], 中的相同，即图像的较短边长为 $s = 600$ 像素。多尺度训练/测试最早在 [12, 7]中被提出，该方法是从特征金字塔中选取特定尺度来实现；而在 [33] 中，则通过使用最大值输出算法来开展此类检测。在目前的实现中，我们按照 [33]的要求进行了多尺度检测，但由于时间有限，并未开展多尺度训练。此外，我们目前仅对Fast R-CNN阶段进行了多尺度检测（尚未针对区域提议网络阶段进行）。利用训练好的模型，我们会在图像金字塔上计算卷积特征图，此时图像的较短边长为 $s \in \{200, 400, 600, 800, 1000\}$ 。

training data	COCO train		COCO trainval	
test data	COCO val		COCO test-dev	
mAP	@.5	@[.5, .95]	@.5	@[.5, .95]
baseline Faster R-CNN (VGG-16)	41.5	21.2		
baseline Faster R-CNN (ResNet-101)	48.4	27.2		
+box refinement	49.9	29.9		
+context	51.1	30.0	53.3	32.2
+multi-scale testing	53.8	32.5	55.7	34.9
ensemble			59.0	37.4

Table 9. Object detection improvements on MS COCO using Faster R-CNN and ResNet-101.

system	net	data	mAP	areo	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
baseline	VGG-16	07+12	73.2	76.5	79.0	70.9	65.5	52.1	83.1	84.7	86.4	52.0	81.9	65.7	84.8	84.6	77.5	76.7	38.8	73.6	73.9	83.0	72.6
baseline	ResNet-101	07+12	76.4	79.8	80.7	76.2	68.3	55.9	85.1	85.3	89.8	56.7	87.8	69.4	88.3	88.9	80.9	78.4	41.7	78.6	79.8	85.3	72.0
baseline+++	ResNet-101	COCO+07+12	85.6	90.0	89.6	87.8	80.8	76.1	89.9	89.9	89.6	75.5	90.0	80.7	89.6	90.3	89.1	88.7	65.4	88.1	85.6	89.0	86.8

Table 10. Detection results on the PASCAL VOC 2007 test set. The baseline is the Faster R-CNN system. The system “baseline+++” include box refinement, context, and multi-scale testing in Table 9.

system	net	data	mAP	areo	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
baseline	VGG-16	07++12	70.4	84.9	79.8	74.3	53.9	49.8	77.5	75.9	88.5	45.6	77.1	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
baseline	ResNet-101	07++12	73.8	86.5	81.6	77.2	58.0	51.0	78.6	76.6	93.2	48.6	80.4	59.0	92.1	85.3	84.8	80.7	48.1	77.3	66.5	84.7	65.6
baseline+++	ResNet-101	COCO+07++12	83.8	92.1	88.4	84.8	75.9	71.4	86.3	87.8	94.2	66.8	89.4	69.2	93.9	91.9	90.9	89.6	67.9	88.2	76.8	90.3	80.0

Table 11. Detection results on the PASCAL VOC 2012 test set (<http://host.robots.ox.ac.uk:8080/leaderboard/displaylb.php?challengeid=11&compid=4>). The baseline is the Faster R-CNN system. The system “baseline+++” include box refinement, context, and multi-scale testing in Table 9.

We select two adjacent scales from the pyramid following [33]. RoI pooling and subsequent layers are performed on the feature maps of these two scales [33], which are merged by maxout as in [33]. Multi-scale testing improves the mAP by over 2 points (Table 9).

Using validation data. Next we use the 80k+40k trainval set for training and the 20k test-dev set for evaluation. The test-dev set has no publicly available ground truth and the result is reported by the evaluation server. Under this setting, the results are an mAP@.5 of 55.7% and an mAP@[.5, .95] of 34.9% (Table 9). This is our single-model result.

Ensemble. In Faster R-CNN, the system is designed to learn region proposals and also object classifiers, so an ensemble can be used to boost both tasks. We use an ensemble for proposing regions, and the union set of proposals are processed by an ensemble of per-region classifiers. Table 9 shows our result based on an ensemble of 3 networks. The mAP is 59.0% and 37.4% on the test-dev set. *This result won the 1st place in the detection task in COCO 2015.*

PASCAL VOC

We revisit the PASCAL VOC dataset based on the above model. With the single model on the COCO dataset (55.7% mAP@.5 in Table 9), we fine-tune this model on the PASCAL VOC sets. The improvements of box refinement, context, and multi-scale testing are also adopted. By doing so

	val2	test
GoogLeNet [44] (ILSVRC’ 14)	-	43.9
our single model (ILSVRC’ 15)	60.5	58.8
our ensemble (ILSVRC’ 15)	63.6	62.1

Table 12. Our results (mAP, %) on the ImageNet detection dataset. Our detection system is Faster R-CNN [32] with the improvements in Table 9, using ResNet-101.

we achieve 85.6% mAP on PASCAL VOC 2007 (Table 10) and 83.8% on PASCAL VOC 2012 (Table 11)⁶. The result on PASCAL VOC 2012 is 10 points higher than the previous state-of-the-art result [6].

ImageNet Detection

The ImageNet Detection (DET) task involves 200 object categories. The accuracy is evaluated by mAP@.5. Our object detection algorithm for ImageNet DET is the same as that for MS COCO in Table 9. The networks are pre-trained on the 1000-class ImageNet classification set, and are fine-tuned on the DET data. We split the validation set into two parts (val1/val2) following [8]. We fine-tune the detection models using the DET training set and the val1 set. The val2 set is used for validation. We do not use other ILSVRC 2015 data. Our single model with ResNet-101 has

⁶<http://host.robots.ox.ac.uk:8080/anonymous/30J40J.html>, submitted on 2015-11-26.

训练数据	COCO训练集		COCO训练验证集	
测试数据	COCO验证集		COCO测试开发集	
mAP	@.5	@[.5, .95]	@.5	@[.5, .95]
基准模型 Faster R-CNN (VGG-16)	41.5	21.2		
基准模型 Faster R-CNN (ResNet-101网络)	48.4	27.2		
+框精炼	49.9	29.9		
+上下文信息	51.1	30.0	53.3	32.2
+多尺度检测	53.8	32.5	55.7	34.9
集成模型			59.0	37.4

表9：使用Faster R-CNN与ResNet-101在MS COCO数据集上进行的物体检测性能提升情况。

系统	net	data	mAP	areo	bike	bird	boat	瓶子	bus	car	cat	椅子	cow	桌子	dog	马	电动自行车、人、植物、羊	sofa	火车	tv			
基准模型	VGG-16网络	07+12	73.2	76.5	79.0	70.9	65.5	52.1	83.1	84.7	86.4	52.0	81.9	65.7	84.8	84.6	77.5	76.7	38.8	73.6	73.9	83.0	72.6
基准模型	ResNet-101网络	07+12	76.4	79.8	80.7	76.2	68.3	55.9	85.1	85.3	89.8	56.7	87.8	69.4	88.3	88.9	80.9	78.4	41.7	78.6	79.8	85.3	72.0
基准模型+++	ResNet-101网络	COCO+07+12	85.6	90.0	89.6	87.8	80.8	76.1	89.9	89.9	89.6	75.5	90.0	80.7	89.6	90.3	89.1	88.7	65.4	88.1	85.6	89.0	86.8

表10. PASCAL VOC 2007测试集上的检测结果。基准模型为Faster R-CNN系统。表9中列出的 “baseline+++” 模型包含了框精修、上下文信息处理以及多尺度测试功能。

系统	net	data	mAP	areo	bike	bird	boat	瓶子	bus	car	cat	椅子	cow	桌子	dog	马	电动自行车、人、植物、羊	sofa	火车	tv			
基准模型	VGG-16网络	07++12	70.4	84.9	79.8	74.3	53.9	49.8	77.5	75.9	88.5	45.6	77.1	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
基准模型	ResNet-101网络	07++12	73.8	86.5	81.6	77.2	58.0	51.0	78.6	76.6	93.2	48.6	80.4	59.0	92.1	85.3	84.8	80.7	48.1	77.3	66.5	84.7	65.6
基准模型+++	ResNet-101	COCO+07++12	83.8	92.1	88.4	84.8	75.9	71.4	86.3	87.8	94.2	66.8	89.4	69.2	93.9	91.9	90.9	89.6	67.9	88.2	76.8	90.3	80.0

表11. PASCAL VOC 2012测试集上的检测结果（网址：<http://host.robots.ox.ac.uk:8080/leaderboard/displaylb.php?challengeid=11&compid=4>）。其中基准模型为Faster R-CNN系统，而 “baseline+++” 模型则在表9中融合了框精炼、上下文信息以及多尺度检测技术。

我们首先根据[33]从特征金字塔中选取两个相邻的尺度。接着会对这两个尺度的特征图分别进行RoI池化处理，并由最大值输出算法将它们合并，操作方式与[33]中的描述一致。研究表明，多尺度检测能使mAP值提升超过2个百分点（见表9）。

正在使用验证数据。接下来，我们采用80k+40k规模的训练验证集进行模型训练，再用20k规模的测试开发集进行性能评估。该测试开发集没有公开的可用于对照的真实标签，相关评估结果由评估服务器生成。在此设置下，模型的mAP@.5值为55.7%，mAP@[.5, .95] 值为34.9%（见表9），这便是仅使用单个模型的测试结果。

集成模型。在Faster R-CNN架构中，系统同时需要学习区域候选框和物体分类器，因此可以通过集成模型来提升这两项任务的性能。我们首先使用集成模型来生成区域候选框，随后再通过由区域级分类器构成的集成体对所有候选框进行进一步处理。表9展示了基于3个网络构成的集成模型所获得的测试结果：在测试开发集上的mAP值分别为59.0%和37.4%。这一成绩曾在*COCO 2015*的检测任务中夺得第一名。

PASCAL VOC数据集

基于上述模型，我们重新在PASCAL VOC数据集上进行了实验。在COCO数据集上使用单一模型时，其mAP@.5值为55.7%（见表9），而我们将该模型在PASCAL VOC数据集上进行微调，并采用了框精炼、上下文信息分析以及多尺度检测等优化手段。通过这些方法，

	val2	test
GoogLeNet算法（ILSVRC 2014竞赛）	-	43.9
我们的单一模型（ILSVRC’ 15竞赛）	60.5	58.8
我们的集成模型（ILSVRC’ 15竞赛）	63.6	62.1

表12. 我们在ImageNet检测数据集上的测试结果（mAP值，%）。我们的检测系统为经过表9中所列改进的FasterR-CNN [32]，该系统采用了ResNet-101网络。

我们在PASCAL VOC 2007数据集上取得了85.6%的mAP值（见表10），在PASCAL VOC 2012数据集上则达到了83.8%的mAP值（见表11）⁶。这一成绩比之前的当前最佳性能结果高出10个百分点[6]。

ImageNet检测任务

ImageNet检测（DET）任务涵盖了200种物体类别，其准确度是通过mAP@.5指标来评估的。我们用于ImageNet DET任务的物体检测算法与表9中所展示的MS COCO任务算法相同。这些神经网络首先在包含1000个类别的ImageNet分类数据集上进行预训练，之后再在DET任务专用数据集上做微调。我们按照[8]的方法将验证集分为两部分（val1/val2）。检测模型通过DET训练集和val1集进行微调，而val2集则用于最终验证。我们没有使用其他2015年ILSVRC竞赛的数据。我们所采用的基于ResNet-101的单一模型便是如此构建的。

⁶<http://host.robots.ox.ac.uk:8080/anonymous/30J40J.html>，提交时间为2015年11月26日。

LOC method	LOC network	testing	LOC error on GT CLS	classification network	top-5 LOC error on predicted CLS
VGG’s [41]	VGG-16	1-crop	33.1 [41]		
RPN	ResNet-101	1-crop	13.3		
RPN	ResNet-101	dense	11.7		
RPN	ResNet-101	dense		ResNet-101	14.4
RPN+RCNN	ResNet-101	dense		ResNet-101	10.6
RPN+RCNN	ensemble	dense		ensemble	8.9

Table 13. Localization error (%) on the ImageNet validation. In the column of “LOC error on GT class” ([41]), the ground truth class is used. In the “testing” column, “1-crop” denotes testing on a center crop of 224×224 pixels, “dense” denotes dense (fully convolutional) and multi-scale testing.

58.8% mAP and our ensemble of 3 models has 62.1% mAP on the DET test set (Table 12). *This result won the 1st place in the ImageNet detection task in ILSVRC 2015*, surpassing the second place by **8.5 points** (absolute).

C. ImageNet Localization

The ImageNet Localization (LOC) task [36] requires to classify and localize the objects. Following [40, 41], we assume that the image-level classifiers are first adopted for predicting the class labels of an image, and the localization algorithm only accounts for predicting bounding boxes based on the predicted classes. We adopt the “per-class regression” (PCR) strategy [40, 41], learning a bounding box regressor for each class. We pre-train the networks for ImageNet classification and then fine-tune them for localization. We train networks on the provided 1000-class ImageNet training set.

Our localization algorithm is based on the RPN framework of [32] with a few modifications. Unlike the way in [32] that is category-agnostic, our RPN for localization is designed in a *per-class* form. This RPN ends with two sibling 1×1 convolutional layers for binary classification (*cls*) and box regression (*reg*), as in [32]. The *cls* and *reg* layers are both in a *per-class* from, in contrast to [32]. Specifically, the *cls* layer has a 1000-d output, and each dimension is *binary logistic regression* for predicting being or not being an object class; the *reg* layer has a 1000×4-d output consisting of box regressors for 1000 classes. As in [32], our bounding box regression is with reference to multiple translation-invariant “anchor” boxes at each position.

As in our ImageNet classification training (Sec. 3.4), we randomly sample 224×224 crops for data augmentation. We use a mini-batch size of 256 images for fine-tuning. To avoid negative samples being dominate, 8 anchors are randomly sampled for each image, where the sampled positive and negative anchors have a ratio of 1:1 [32]. For testing, the network is applied on the image fully-convolutionally.

Table 13 compares the localization results. Following [41], we first perform “oracle” testing using the ground truth class as the classification prediction. VGG’s paper [41] re-

method	top-5 localization err	
	val	test
OverFeat [40] (ILSVRC’13)	30.0	29.9
GoogLeNet [44] (ILSVRC’14)	-	26.7
VGG [41] (ILSVRC’14)	26.9	25.3
ours (ILSVRC’15)	8.9	9.0

Table 14. Comparisons of localization error (%) on the ImageNet dataset with state-of-the-art methods.

ports a center-crop error of 33.1% (Table 13) using ground truth classes. Under the same setting, our RPN method using ResNet-101 net significantly reduces the center-crop error to 13.3%. This comparison demonstrates the excellent performance of our framework. With dense (fully convolutional) and multi-scale testing, our ResNet-101 has an error of 11.7% using ground truth classes. Using ResNet-101 for predicting classes (4.6% top-5 classification error, Table 4), the top-5 localization error is 14.4%.

The above results are only based on the *proposal network* (RPN) in Faster R-CNN [32]. One may use the *detection network* (Fast R-CNN [7]) in Faster R-CNN to improve the results. But we notice that on this dataset, one image usually contains a single dominate object, and the proposal regions highly overlap with each other and thus have very similar RoI-pooled features. As a result, the image-centric training of Fast R-CNN [7] generates samples of small variations, which may not be desired for stochastic training. Motivated by this, in our current experiment we use the original R-CNN [8] that is RoI-centric, in place of Fast R-CNN.

Our R-CNN implementation is as follows. We apply the per-class RPN trained as above on the training images to predict bounding boxes for the ground truth class. These predicted boxes play a role of class-dependent proposals. For each training image, the highest scored 200 proposals are extracted as training samples to train an R-CNN classifier. The image region is cropped from a proposal, warped to 224×224 pixels, and fed into the classification network as in R-CNN [8]. The outputs of this network consist of two sibling fc layers for *cls* and *reg*, also in a per-class form. This R-CNN network is fine-tuned on the training set using a mini-batch size of 256 in the RoI-centric fashion. For testing, the RPN generates the highest scored 200 proposals for each predicted class, and the R-CNN network is used to update these proposals’ scores and box positions.

This method reduces the top-5 localization error to 10.6% (Table 13). This is our single-model result on the validation set. Using an ensemble of networks for both classification and localization, we achieve a top-5 localization error of 9.0% on the test set. This number significantly outperforms the ILSVRC 14 results (Table 14), showing a 64% relative reduction of error. *This result won the 1st place in the ImageNet localization task in ILSVRC 2015*.

LOC 方法	LOC 神经网络	测试	位置实体误差在真实类别标注上	分类神经网络	前五位置误差基于预测类别标注
VGG的 [41]	VGG-16单裁剪版本		33.1 [41]		
RPN	ResNet-101单裁剪版本		13.3		
RPN	ResNet-101密集连接版本		11.7		
RPN	ResNet-101密集连接版本			ResNet-101网络	14.4
RPN+RCNN结合	ResNet-101密集连接版本			ResNet-101网络	10.6
RPN+RCNN	集成模型	密集层		集成模型	8.9

表13. ImageNet验证集上的定位任务误差 (%)。在“真实类别的LOC误差” ([41]) 这一列中，所使用的为真实标签；而在“测试”一列中，“1-crop”表示在尺寸为 224×224 像素的图像中心区域进行测试，“dense”则表示采用密集型（全卷积）及多尺度检测方式。

在DET测试集上，该模型的mAP值为58.8%，而我们构建的3个模型组成的集成模型的mAP值则为62.1%（见表12）。这一成绩在2015年ILSVRC竞赛的*ImageNet*检测任务中获得了第一名，其优势幅度比第二名高出**8.5分**（绝对值）。

C. ImageNet定位任务

ImageNet定位任务（LOC）[36] 要求对图像中的对象进行分类与定位。基于 [40, 41], 的思路，我们首先使用图像级分类器来预测图像的分类标签，而定位算法则仅依据这些预测的类别来生成边界框。为此我们采用了“每类回归”（PCR）策略 [40, 41], , 即为每个类别单独训练一个边界框回归模型。我们先对这些神经网络进行ImageNet分类任务的预训练，之后再针对定位任务对其微调，所有训练都在提供的1000类ImageNet训练集上进行。

我们的定位算法是在 [32] 的RPN框架基础上稍作修改后构建的。与那种不考虑类别差异的实现方式不同，用于定位的RPN是我们按每类的方式设计的。该RPN的最后部分包含两个相邻的 1×1 卷积层，分别用于执行二元分类 (*cls*) 和边界框回归 (*reg*)，这一设计与 [32]中的方案一致。*cls* 层和*reg* 层同样采用每类的结构，这与 [32]中的设计有所不同。具体而言，*cls* 层的输出维度为1000，每个维度都通过二元逻辑回归来判断该位置是否属于某个物体类别；而*reg* 层的输出维度为 1000×4，其中包含了针对1000个类别的边界框回归模型。和 [32],中所述的一样，我们的边界框回归方法是参考每个位置处多个具有平移不变性的“锚点”框来实现的。

与我们在ImageNet分类训练中采用的方式相同（见 3.4节），我们会随机抽取 224×224 张图像作为样本进行数据增强。在微调阶段，我们使用256张图像作为小批量大小。为避免负面样本占据主导地位，每张图像会随机选取8个锚点，且这些被选中的正负锚点比例约为1:1 [32]。在测试阶段，该神经网络会对图像进行全卷积处理。

表13展示了各项定位结果的对比情况。如[41], 所述，我们首先采用真实类别标签作为分类预测结果，进而进行所谓的“神谕测试”。VGG在相关论文 [41] 中报告的

方法	前五名定位误差	
	val	test
OverFeat [40] (ILSVRC 2013竞赛)	30.0	29.9
GoogLeNet [44] (ILSVRC 2014竞赛)	-	26.7
VGG [41] (ILSVRC 2014竞赛)	26.9	25.3
我们的成果 (ILSVRC’ 15)	8.9	9.0

表14. 表14展示了使用最先进方法在ImageNet数据集上进行的定位误差 (%) 对比情况。

在使用真实类别标签的情况下，该方法的中心裁剪误差为 33.1%（见表13）。在相同条件下，我们基于ResNet-101网络开发的区域提议网络方法将这一中心裁剪误差显著降低至 13.3%。这一对比充分体现了我们所提出框架的出色性能。在采用密集型（全卷积型）检测以及多尺度检测方案时，使用真实类别标签的ResNet-101网络的误差仅为11.7%。而当利用ResNet-101网络进行类别预测时（表4显示其前五名分类误差为4.6%），其对应的前五名定位误差则为14.4%。

上述结果仅基于Faster R-CNN [32]中的提议网络（即RPN）得出。人们也可以采用Faster R-CNN中的检测网络（Fast R-CNN [7]）来提升结果。但需要注意的是，在该数据集中，一张图像通常只包含一个主导物体，且各个提议区域彼此高度重叠，因此它们生成的基于ROI池化的特征也非常相似。正因如此，Fast R-CNN [7]的以图像为中心的训练方式只能产生变化较小的样本，而这并不符合随机训练的需求。基于这一考虑，我们在当前的实验中采用了以ROI为中心的原始R-CNN [8], 而非Fast R-CNN。

我们的R-CNN实现方案如下：首先在训练图像上应用前述针对各类别训练好的RPN，为真实类别预测边界框，这些预测框可作为与类别相关的提议区域。对于每张训练图像，系统会选取得分最高的200个提议区域作为训练样本，用以训练R-CNN分类器。随后从这些提议区域中裁剪出图像部分，将其映射到 224×224像素尺寸，再像R-CNN [8]那样输入到分类网络中。该网络的输出包含两个用于分别处理类别识别和位置估计任务的全连接层，同样也是按类别独立设计的。之后，这个R-CNN网络会以ROI为中心，在包含256个小批量大小的训练集上进行微调。在测试阶段，RPN会为每个预测出的类别生成得分最高的200个提议区域，随后R-CNN网络则会被用来更新这些提议区域的得分及边界框位置。

该方法将前五名定位误差降低到了10.6%（表13）。这是我们在验证集上使用单模型所取得的成绩。而通过构建用于分类与定位任务的集成模型，在测试集上的前五名定位误差则降至9.0%。这一数值显著优于ILSVRC 2014竞赛的结果（表14），误差降低了64%。这一成果曾在2015年ILSVRC竞赛的*ImageNet*定位任务中荣获第一名。